Computer Science and Engineering

Software Engineering II

2025 – 2026

Requirement Analysis and Specification Document

Best Bike Paths

Authors:

Wojciech Maciej Bukala (11152502)

Giacomo Rigo (11106172)

Mattia Zonzin (10849943)

Professor:

Prof. Elisabetta Di Nitto

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.1.1 Goals . . . 3
1.2 Scope . . . 4
1.2.1 World Phenomena . . . 4
1.2.2 Shared Phenomena . . . 5
1.3 Definitions, Acronyms, Abbreviations . . . 6
1.3.1 Definitions . . . 6
1.3.2 Acronyms . . . 6
1.3.3 Abbreviations . . . 6
1.4 Reference Documents . . . 7
1.5 Revision history . . . 7
1.6 Document Structure . . . 7

2 Overall description **9**
2.1 Product perspective . . . 9
2.1.1 Scenarios . . . 9
2.2 Domain Class Diagrams . . . 10
2.3 State Diagrams . . . 12
2.3.1 Automatic trip registration . . . 12
2.3.2 Weather data acquire . . . 13
2.3.3 Path lifecycle . . . 14
2.4 Obstacle detection . . . 15
2.4.1 User account lifecycle . . . 16
2.5 Product functions . . . 17
2.5.1 General Functions . . . 17
2.5.2 Registered User Functions . . . 17
2.5.3 Any User Functions . . . 18
2.6 User Characteristics . . . 18
2.6.1 Registered User . . . 18
2.6.2 General User . . . 19
2.6.3 BBP Administrator . . . 19
2.7 Assumptions, Dependencies and Constraints . . . 19
2.7.1 Domain Assumptions . . . 19
2.7.2 Dependencies and Constraints . . . 20

3 Specific requirements **21**
3.1 External Interface Requirements . . . 21
3.1.1 User Interface . . . 21
3.1.2 Hardware Interfaces . . . 22

3.2 Functional Requirements . . . 22
3.2.1 Requirements Mapping . . . 24
3.2.2 Use cases Diagrams . . . 26
3.2.3 Use cases . . . 28
3.2.4 Use Cases' Sequence Diagrams . . . 37
3.3 Performance Requirements . . . 50
3.4 Design Constraints . . . 50
3.4.1 Standards compliance . . . 50
3.4.2 Hardware Limitations . . . 51
3.5 Software System Attributes . . . 51
3.5.1 Reliability . . . 51
3.5.2 Availability . . . 51
3.5.3 Security . . . 51
3.5.4 Maintainability . . . 52
3.5.5 Portability . . . 52

4 Formal analysis using alloy **53**
4.1 Alloy model . . . 53
4.2 Assertions . . . 61
4.2.1 Results of Assertions . . . 63
4.3 Predicates . . . 63
4.3.1 Results . . . 66

5 Effort spent **69**

1 Introduction

1.1 Purpose

In recent years, cycling has become very popular as a green mode of transport and for leisure. Increasing environmental awareness, high fuel prices, and the need for healthier lifestyles have pushed many people to opt for bicycles not only for their daily commutes but also for tourist travel and leisure purposes. Cities and small communities across the world are increasingly investing in cycling infrastructure, but bike riders still struggle to find safe, efficient, and well-maintained cycling routes. Road conditions, traffic, and environmental factors such as weather can significantly impact not only the accessibility of bike paths but also the fun of riding a bike. Best Bike Paths (BBP) system aims to be a pioneering platform that combines community participation and technological potential to improve the cycling experience. It allows users to record their trips, find all the information on bike paths they are interested in, and share their experience with the wider cycling community. In BBP, community input and smart data analysis come together to make cycling safer, more accessible, and more fun for everyone.

1.1.1 Goals

G1: Allow any user with access to the platform to register and create a personal account to access functionalities reserved for registered users.

G2: Allow a registered user to record, store and publish the details of their bike trips such as total distance, duration, and average speed enriched, when available, with meteorological information.

G3: Allow registered users to contribute to updating information about cycling routes by providing data and reports regarding the quality and safety of the paths.

G4: Allow any user to find optimal and relevant bike paths between a specified origin and destination.

G5: Allow any user to rely on accurate and aggregated information about the condition and quality of bike paths, derived from multiple user contributions.

1.2 Scope

The principal function of the Best Bike Paths (BBP) network is to operate as a platform that enables interaction and data interchange within the BBP ecosystem, that apart from the users, includes the system itself. BBP users are primarily cyclists who can be either registered or unregistered. Registered users are allowed to do everything in the system, like recording their trips, keeping track of their stats, and even giving information about the bike routes. Meanwhile, unregistered users are only given access to exploring and viewing the information of the routes but cannot add or modify any data. To ensure the reliability and accuracy of the collected information, the system also interacts with external services such as meteorological data providers, which supply environmental information like temperature, wind speed, and weather conditions. The BBP system supports two principal modes for the actual data entrance: manual and automated. In the former scenario, users can fill in the missing pieces of the story of the local bike route such as names of the streets, how the route is, and if there is any type of obstruction. In case of an automated mode, the system can simply reach out to the user's mobile device and gather all the required data through GPS and different sensors such as accelerometers and gyroscopes that help in detecting the surface and can hence use this info to determine if there are some cracks or holes on the road. The information gathered automatically must be confirmed or corrected by the user before becoming publicly available to ensure accuracy and avoid false positives. All registered users interact with BBP through personalized accounts, which allow them to manage their trips, store personal cycling data, and choose whether to make their path information public. The system must collect, unify, and validate user data (based on policies defined by the BBP Administrator) to ensure published information is complete and coherent. Therefore, the bike routes that are presented to all users are based on the most recent and accurate data. To sum up, BBP is an absorbent platform that performs not only the function of bridging the gap between cyclists and the correct information on the routes but also the promotion of a collaborative environment whereby users contribute to improving the overall cycling experience.

1.2.1 World Phenomena

WP1: Cyclists (Registered and Unregistered Users) travel along bike paths and identify the origin and destination points of their trips.

WP2: Bike paths consist of dedicated tracks or roads where traffic conditions and speed limits are compatible with the average cycling speed.

WP3: Bike paths have physical characteristics such as surface type and width

WP4: Weather conditions, including temperature, wind, and precipitation, influence cyclists' performance and overall experience.

WP5: Registered Users, during their trips, carry mobile devices capable of detecting motion, position, and orientation through sensors (GPS, accelerometer, and gyroscope).

WP6: The cycling activity of a Registered User can be inferred from motion data, such as speed and continuity of movement.

WP7: Registered Users describe and record manually the details and condition of bike paths.

WP8: Registered Users validate or correct automatically detected information about obstacles before publication.

WP9: BBP Administrator defines the business policies for merging, maintaining and verifying data collected from users.

1.2.2 Shared Phenomena

The following shared phenomena involve interactions controlled by the world:

SP1: Registered Users manually input details about a bike path, including streets, path condition, and possible obstacles.

SP2: The system receives GPS data, speed, and sensor readings (accelerometer, gyroscope) from the user's mobile device during a cycling activity.

SP3: Registered Users confirm, edit, or reject automatically detected information about a path before publishing it.

SP4: Registered Users submit verified path information for publication and community access.

SP5: Any user (Registered or Unregistered) specifies an origin and a destination to request a bike path recommendation.

SP6: Registered Users browse, view, and manage their stored trips and related data.

The following shared phenomena involve interactions controlled by the machine:

SP7: The system reconstructs and displays the user's tracked path on a map for review.

SP8: The system retrieves weather data (temperature, wind, conditions) from an external meteorological service.

SP9: The system displays available paths between the selected points, ranked by condition and effectiveness.

SP10: The system provides users with detailed statistics and performance metrics for their recorded trips (e.g., total distance, duration, average speed).

SP11: The BBP Administrator defines (or updates) the parameters that the system uses to aggregate and verify path information.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

The following terms are used throughout this document with the meanings specified below.

- **User**: Any person interacting with the BBP platform. Users can be either registered or unregistered.
- **BBP Administrator**: A user representing the Bikers' Association, responsible for configuring system policies and moderating community-provided content.
- **Trip/Ride**: A cycling activity performed by a registered user.
- **Bike Path/Cycling Route**: A route suitable for cycling, consisting of either dedicated cycling infrastructure or road segments with limited traffic and speed conditions compatible with bicycles.
- **Automatic Recording**: A mode in which the system collects trip data using a user's mobile device sensors.
- **Manual Path Insertion**: The process by which a registered user manually inputs details about a cycling route.
- **Route Ranking**: The mechanism through which available cycling routes are presented to users in an ordered manner based on factors such as surface quality, road safety, community feedback, and route efficiency.
- **Obstacle**: Any road condition or irregularity that may affect cycling safety or comfort, such as potholes.

1.3.2 Acronyms

- **RASD**: Requirements Analysis and Specification Document
- **BBP**: Best Bike Paths
- **GPS**: Global Positioning System
- **API**: Application Programming Interface

1.3.3 Abbreviations

- **G**: Goal
- **FR**: Functional Requirement
- **NFR**: Non-Functional Requirement
- **DA**: Domain Assumption

- **WP**: World Phenomena
- **SP**: Shared Phenomena

1.4 Reference Documents

The content of this document, along with the related analysis and requirements, is based on the following sources:

- The official specification of the 2024/2025 Requirements Engineering and Design Project for the Software Engineering II course.
- The lecture slides and learning materials available on the WeBeep platform for the Software Engineering II course.

1.5 Revision history

- 12-12-2025: Version 1.0
- 18-12-2025: Version 2.0

1.6 Document Structure

1. **Introduction**
   Provides a brief description of the Best Bike Paths (BBP) project based on the given specification. It outlines the main objectives of the product and defines its overall scope.
2. **Overall Description**
   Presents a detailed overview of the system, including its context, actors, and main functionalities. It also describes real-world scenarios and offers a high-level technical analysis of the system's architecture and interactions.
3. **Specific Requirements**
   Contains a comprehensive list of functional and non-functional requirements, along with their relationships to goals and domain assumptions. This section specifies what the system must do to satisfy stakeholder needs.
4. **Formal Analysis Using Alloy**
   Provides a formal representation of a subset of the modeled world and shared phenomena through Alloy. The analysis aims to identify and resolve possible inconsistencies among assumptions, goals, and requirements.

5. **Effort Spent**
   Reports the contribution and time dedicated by each member of the group to the realization of this document.

6. **References**
   Lists all external sources, documents, and tools used in the preparation and writing of this RASD.

2 Overall description

2.1 Product perspective

2.1.1 Scenarios

- **User registration**
  A new user who does not yet have an account wants to create one, to save their bike trips and share them with the community. The user enters the required information: email, password, and nickname. The user may also provide optional profile details, such as image, age, gender or link social medias. If the credentials meet the system requirements, the account is created.

- **User logs in**
  A user who already has an account wants to log in. The user navigates to the login page, enters their credentials, and confirms. Once access is granted, the user can use registered-only features, such as publishing paths or browsing personal history. If the credentials are incorrect, the user can retry or reset the password.

- **Registered user inserts path manually**
  A registered and logged-in user selects the *Insert path manually* option. In the insertion view, the user specifies the streets and their status. The user can optionally add obstacles such as potholes or construction works. The saved path can be published later.

- **Registered user records path automatically**
  A registered and logged-in user starts recording a path. While biking, the device provides GPS and sensor data. When the user stops recording, the path is saved and linked to the user's account. The recorded path can be confirmed and published later.

- **Registered user confirms automatic recorded path**
  A registered and logged-in user with previously recorded paths can browse them in their history. he user opens a recorded path, reviews the details, and corrects data if needed (e.g., removing false obstacles). The user then confirms the path. Once confirmed, the path becomes eligible for publication. If the user rejects the path, it remains private or can be deleted.

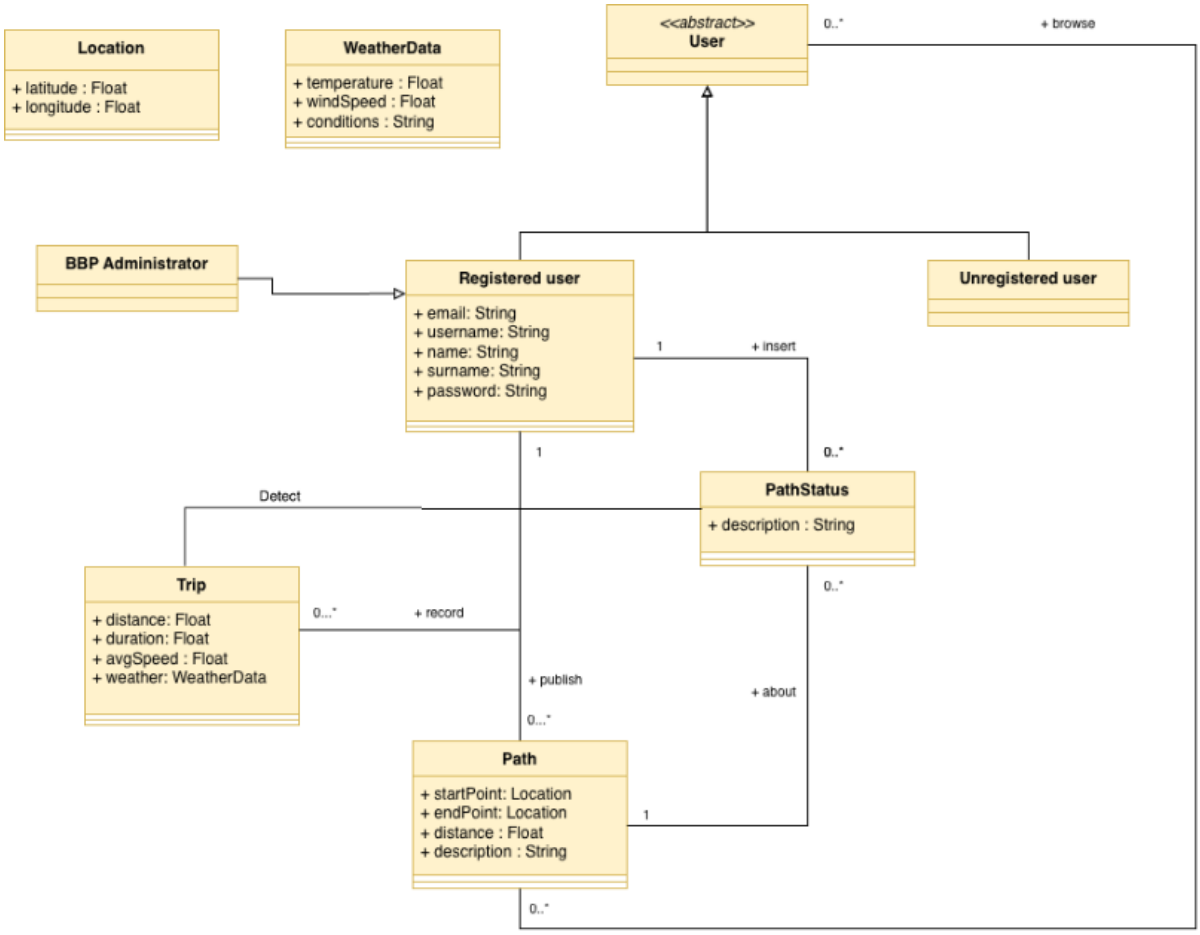
- **Registered user browses history**
  A registered and logged-in user selects the *My history* option from the menu. The user browses the list of saved paths. Paths can be sorted by date, distance, or status. For each path, the user can open details such as distance, duration, or average speed. The user may also delete or edit a path.

- **Registered user publish path**
  A registered and logged-in user chooses one of their own paths from history. The user

decides which path to publish and may add a short description or tags. Once published, the path becomes available to both registered and non-registered users.

- **User browses for bike path**
  Any user, whether registered or not, specifies a starting and ending point for a path. The system provides available paths with their status and ranking. The user browses the list, views details such as distance and condition, and selects the most suitable one. A registered user may also save the chosen path for a later use.

2.2 Domain Class Diagrams



The Domain Class Diagram shown above is a conceptual model of entities and relations between them in the BBP domain. Worth mentioning clarifications:

- The Domain Class Diagram is concentrated on entities that occur in the application domain and models only features of entities without operations. The diagram does not determine the classes for implementation.

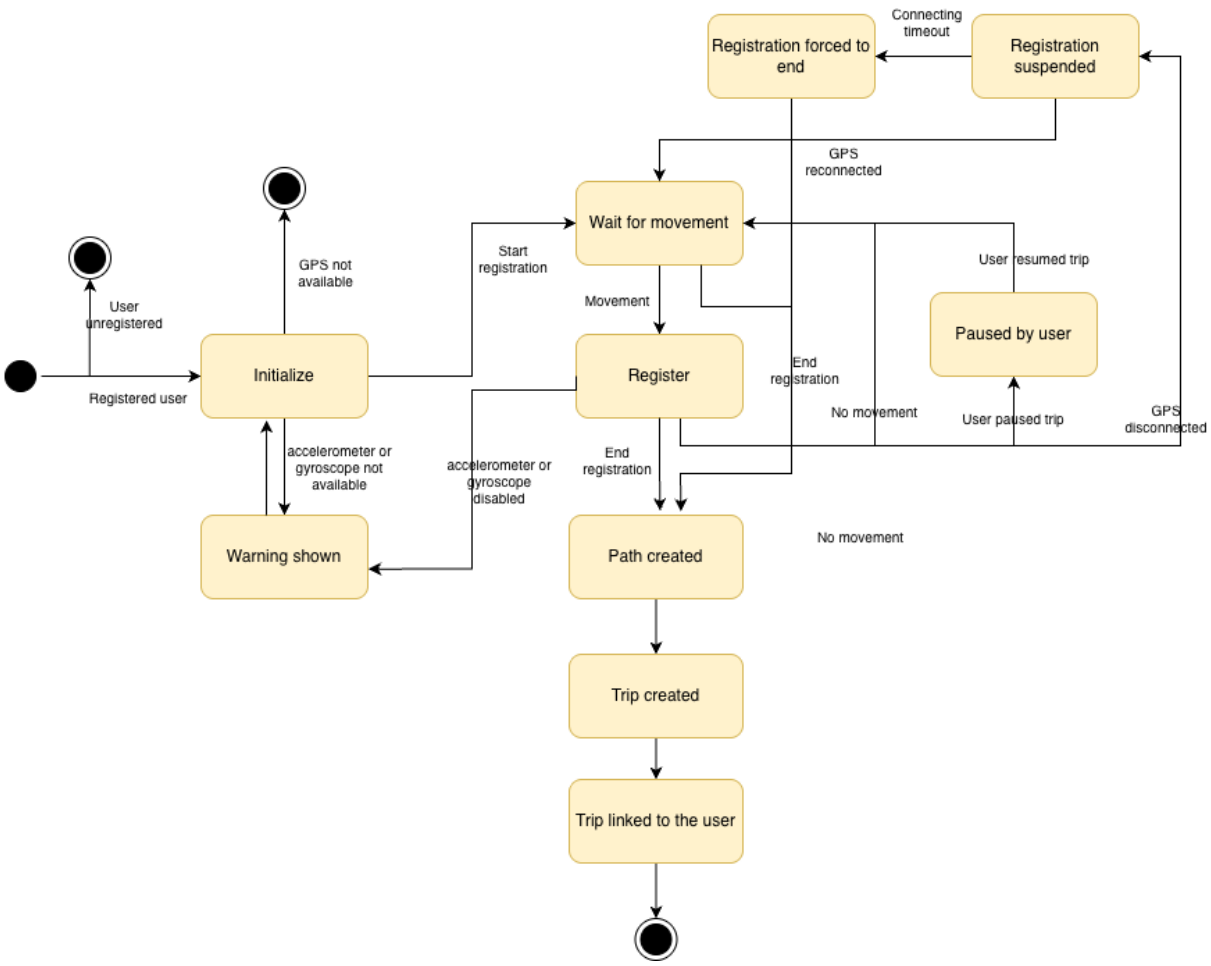
- Distinguishing between `Trip` and `Path` classes is crucial for understanding the concept. `Trip` represents a saved activity of the user. `Path` is the recording of the route in the

physical world; it contains start/end locations to reconstruct the route on a map. `Trip`, contains information about performance metrics, such as distance, duration and average speed.

- `Registered user` registered user can publish multiple `Path` and can record and store many `Trips`.
- A `User` (resgistered or not) can browse for an available bike path stored in the system by specifying an origin and a destination. The user is then able to explore the proposed paths and view their characteristics.
- A `RegisteredUser` can insert multiple `PathStatus` concerning one specific `Path`. The system will need to calculate a path's aggregated score based on its `PathStatuses`.
- A `Trip` may automatically detect some obstacles and report them to `PathStatus`
- `WeatherData` and `Location` are modeled as embedded value types rather than independent entities/classes. They are stored inside their owning objects (e.g. `WeatherData` are stored inside the `Trip` instance). That is why they are not connected with any other entity on the diagram.

2.3 State Diagrams

2.3.1 Automatic trip registration



This state machine represents the trip registration process, so it is strictly connected with the `Trip` class and can initialize `the Path` object during the process. A `Trip` can be initialized only by registered user. If the user is not registered, BBP refuses the registration. At the initialization stage, BBP ensures that GPS localization is available; if not, registration cannot be performed. If the accelerometer or gyroscope is unavailable, the process still takes place; however, the user must be informed that obstacles and other anomalies requiring these sensors will not be recorded.

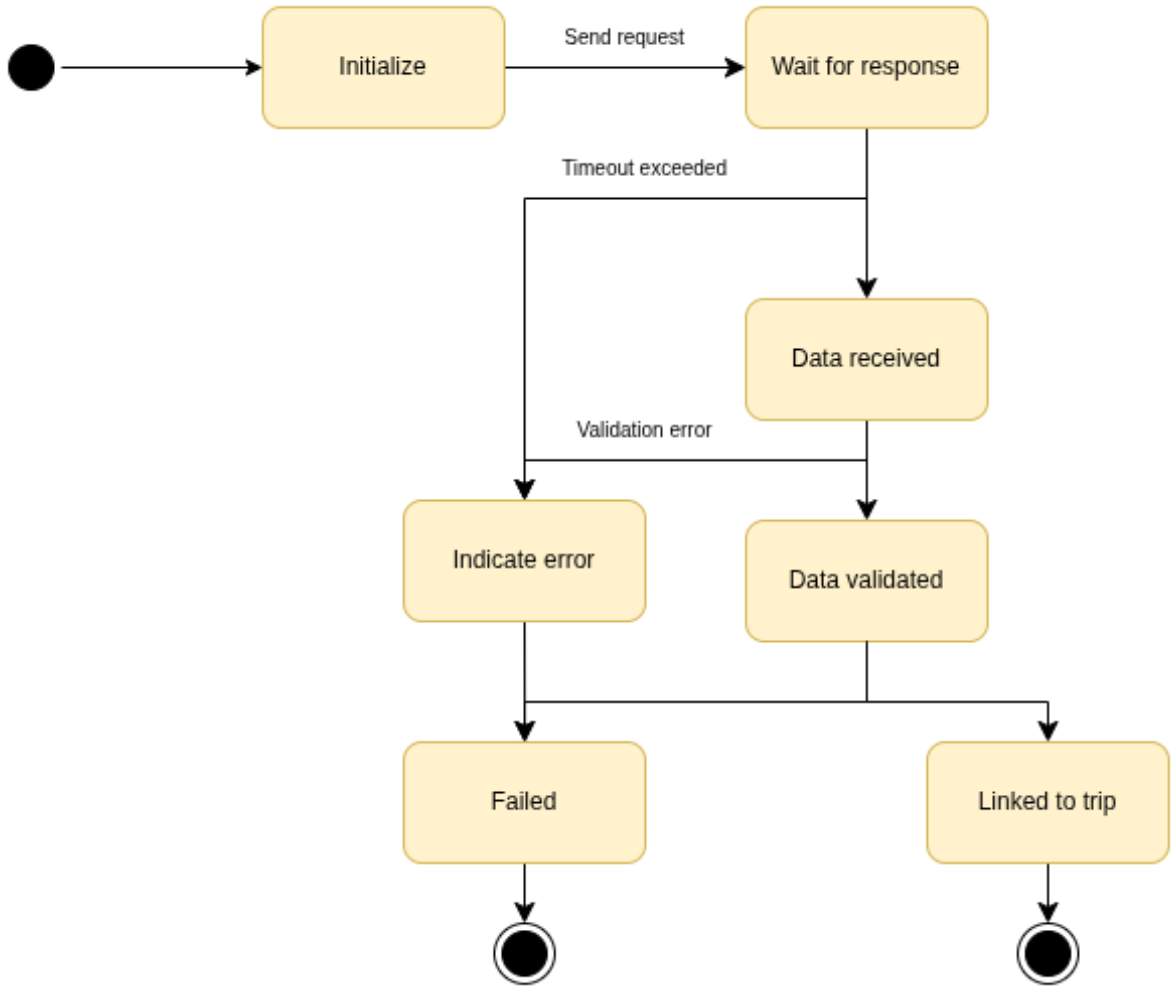
The registration process runs in a loop. At each iteration (timestamp), BBP checks:

1. GPS connection – it is crucial for the process. If the GPS connection is lost, the application restarts and waits for reconnection. If the timeout is reached, the registration ends, and the path and trip are created and linked to the user's profile,

2. movement – based on the calculated speed and changes in localization, BBP detects whether the user is biking. If not, the registration enters the *wait for movement* state. In this state, the user can either resume the trip (by starting to move) or end it.

Each `Trip` is stored and linked to the corresponding `RegisteredUser`, according to the Domain Class Diagram.

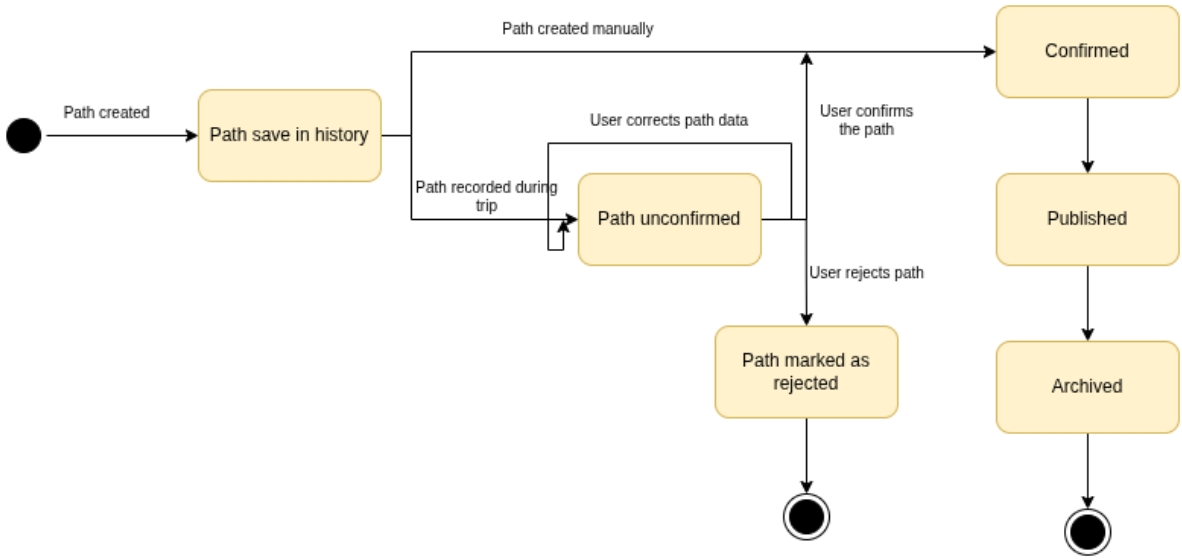
2.3.2 Weather data acquire



The state machine above models the process through which BBP acquires, validates, and associates weather data with a recorded `Trip`.

Once a trip recording is completed, BBP triggers a weather data request. After sending the request, BBP waits for a response for a specified period of time. If the response is not received within this timestamp, the process fails, and the failure is indicated in the trip record. If the response is received correctly and successfully validated, the weather data is linked to the `Trip`.

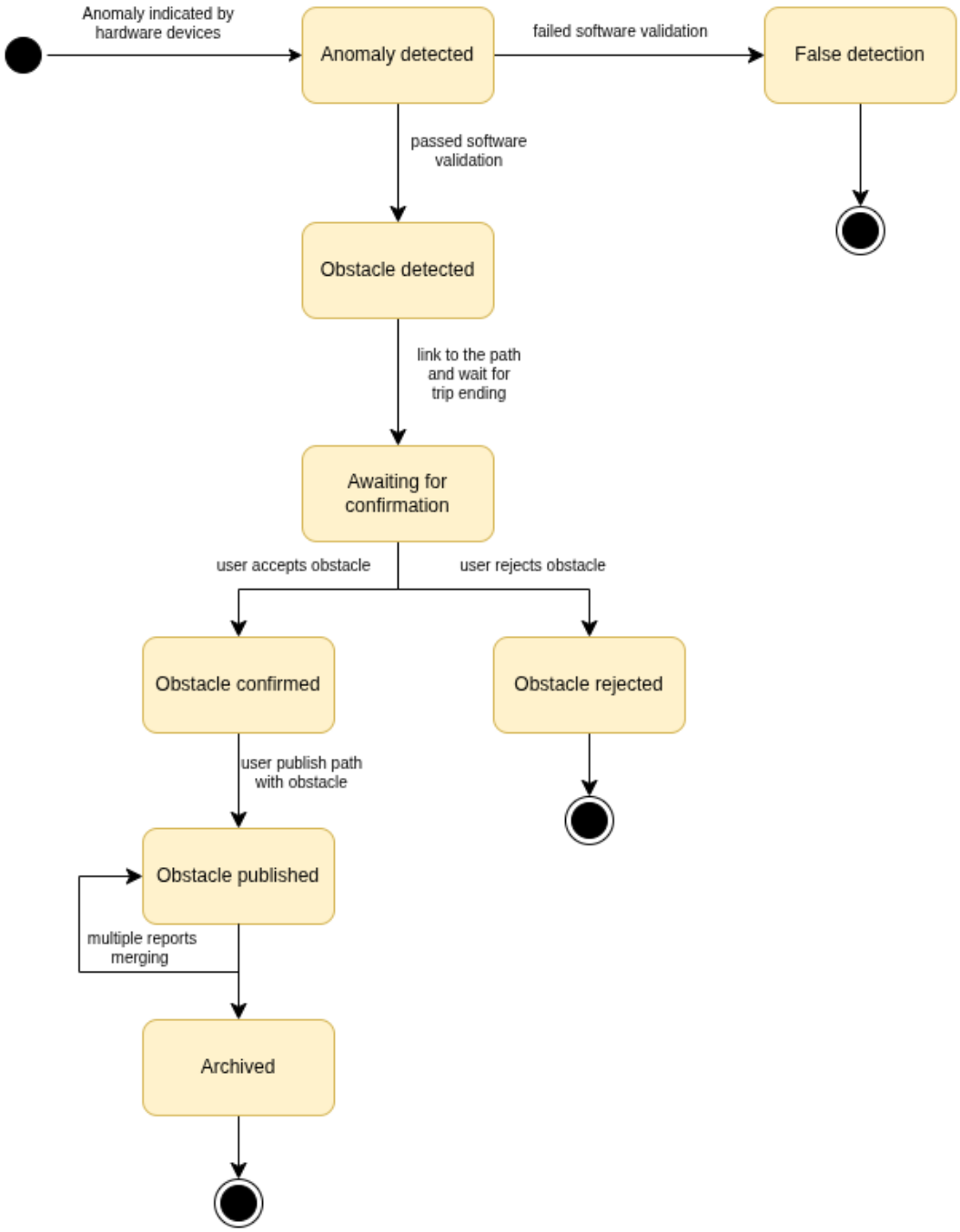
2.3.3 Path lifecycle



The path lifecycle state machine corresponds to the different stages of the `Path` class.

A `Path` can be created and linked to an account either automatically during the trip recording process or inserted manually. If the path is created automatically, it is marked as unconfirmed and can be reviewed by the user — confirmed, modified and confirmed, or rejected. If a `Path` is rejected, it remains stored in memory to indicate the route of the associated `Trip`, but it cannot be published. Confirmed `Paths` can be published, and later the user can decide to archive the `Path`.

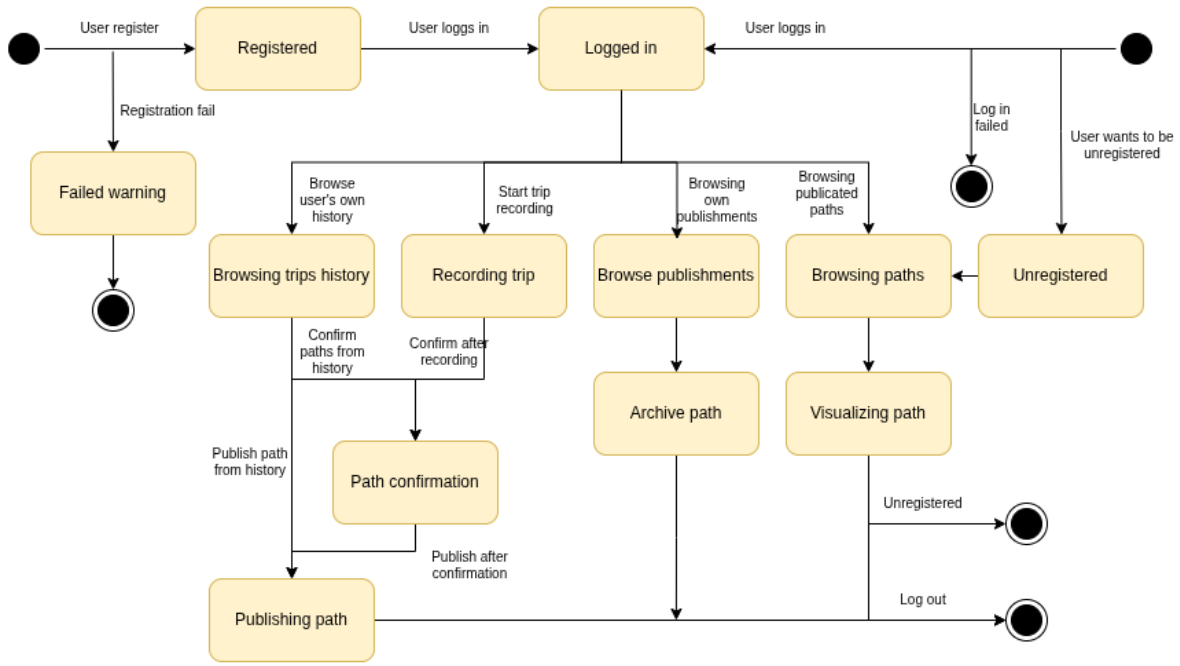
2.4 Obstacle detection



The diagram presented above focuses on the stages of the `Obstacle` class. Note that `Obstacles` cannot exist without the `Paths` to which they are linked. `Obstacles` are detected by device hardware such as an accelerometer or gyroscope during `Trip` recording. The software should be able to perform a primary validation step to reject obvious false detections.

After the recording is completed, each obstacle waits for user confirmation. Once the `Path` containing the `Obstacle` is published, the lifecycles of both become similar. `Obstacles` can have multiple reports and the BBP must determine which is the most accurate.

2.4.1 User account lifecycle



The state diagram above presents the states in which a `User` and a `RegisteredUser` can be. A user can start the registration process. If the provided data does not meet the requirements or matches an already existing account, the process fails, and the user can start over. If the user is successfully registered and logged in, they can start a new trip recording, browse their previous trips in the history panel, or view published paths.

Both newly registered trips and trips from history can be confirmed if they were registered automatically and are marked as unconfirmed. Every confirmed path linked to the account can be published, and every previously published path can be archived. Both `RegisteredUsers` and `Users` can browse publicly available paths and visualize them on the map.

2.5 Product functions

2.5.1 General Functions

Signup
Users sign up by providing their name, surname, nickname, email and a password. After submitting the form, a confirmation email is sent. Only after confirming the email the account becomes active and the user can access BBP's features.

Login
Users log in by typing their registered email and password. If the credentials are correct, the system grants access to the user dashboard; otherwise, it notifies the user of the failed attempt.

Data Aggregation
The system automatically combines multiple reports referring to the same path segment. It applies predefined rules to generate a shared and updated evaluation of path quality.

Retrieve Weather Data
When a trip is recorded, BBP queries an external weather service to retrieve temperature, wind, and weather conditions. These details are then stored together with the user's trip for contextual analysis.

2.5.2 Registered User Functions

Manual Path Insertion
Registered users can manually select streets on a map and assign a condition level. Optional notes or photos can also be added. This information may remain private or be made available to the community.

Automatic Trip Recording
Registered users can start automatic recording before biking. The system uses GPS and motion sensors to track the route and detect possible obstacles automatically while the user cycles.

Trip Review & Confirmation
After an automatic trip, the user reviews the detected events. They can confirm valid obstacles, edit incorrect ones, or delete false detections before saving the trip.

Publish Trip Data
Registered users can publish recorded trip data to share it with the community. Alternatively, trips can be kept private for personal use.

Display Trip Statistics
At the end of the ride, BBP displays basic performance metrics such as total distance, duration,

average speed, and the route on a map. This helps users monitor their cycling activity over time.

Browse Trip History
Registered users can access a personal list of all previously recorded trips, both manual and automatic. they may select any trip to view its complete information, including performance statistics, detected obstacles, and whether it is published or private.

2.5.3 Any User Functions

Search for Bike Paths
Both registered and guest users can input an origin and destination to request possible bike routes between the two. The system processes the request and shows one or more suggested paths.

View Route Information
Users can inspect route details including distance, estimated travel time, path condition ratings and known obstacles. The route is also shown on an interactive map for easier understanding.

Filter Routes
Users can refine route suggestions by applying filters such as avoiding steep hills or preferring smoother road surfaces. This allows cyclists to choose paths that best match their comfort level or cycling preferences.

Browse Pubblicated Paths
Users can search for bike paths published by other users. The system allows filtering by origin and destination, path length, condition level, and recency of reports. Search results show a list of matching public routes, and users can select any of them to view detailed information, including the map, reported obstacles, and overall path condition.

2.6 User Characteristics

2.6.1 Registered User

The Registered User can sign up and log in to the platform. Their profile keeps track of personal trips and contributions made to the community. They can use the application to record and save their trips. For each trip, the system provides statistics . If available, the system also adds weather information from external services. The Registered User is the main source of information about path conditions.

They can add data in two ways:

- Manual Mode: By entering street names, the path status, and information about obstacles.
- Automatic Mode: By letting BBP collect data from their mobile device while they ride.

For automatic data, the user must confirm or correct the information found by the system before it is saved. The user can then decide to "publish" the collected information (both manual and automatic) to share it with the community.

2.6.2 General User

The General User is anyone who uses the system to find information. This role can be filled by a visitor or by a Registered User. The General User does not provide path data; they only use it. They can enter a start point and an end point and ask the system to show the available bike paths on a map. If BBP finds multiple paths, it shows them to the user ranked by a score. This score is calculated by the system based on the path's condition (from community data) and how well the path connects the start and end points.

2.6.3 BBP Administrator

This user is responsible for setting up the data-merging rules. While the system merges data automatically, the Administrator defines the parameters BBP uses to solve conflicts (giving priority to newer data or data with more confirmations). Additionally, the Administrator monitors, reports and has the authority to moderate or remove published information that is clearly wrong, outdated, or inappropriate. This ensures that the path inventory remains reliable.

2.7 Assumptions, Dependencies and Constraints

2.7.1 Domain Assumptions

DA1: Registered Users provide accurate manual path information.

DA2: Registered Users validate and correct their automatically recorded paths before publishing them.

DA3: The BBP Administrator defines consistent data-merging and validation policies.

DA4: All users have access to a stable internet connection, sufficient to interact with the BBP platform and exchange data.

DA5: Registered Users' mobile devices are equipped with functioning and sufficiently precise sensors.

DA6: Sensor-detected data reliably correlate with the presence of real obstacles.

DA7: The external meteorological service is reachable and provides accurate, up-to-date weather data for a given location and time.

DA8: Map and route data provided by external services are available and sufficiently accurate for navigation and visualization.

DA9: Users' devices maintain enough battery life to complete trip recordings and upload collected data.

DA10: The community of users is active enough to ensure that paths are frequently updated and verified.

2.7.2 Dependencies and Constraints

The operation of the Best Bike Paths (BBP) system depends on the availability of several external services and technologies required to make all its functionalities fully operative. In particular, the following services and conditions are necessary:

- **Map Service**: Provides the geographic data and visualization features necessary to display bike paths, compute routes between origin and destination, and represent the overall road network on a map.
- **Meteorological Service**: Supplies weather-related information such as temperature, wind speed, and weather conditions corresponding to specific locations and time intervals. This data enriches trip statistics and helps users better interpret their performance.
- **Mobile Device Sensors**: The automatic trip recording functionality relies on sensors, that are integrated into users' mobile devices. GPS for location tracking, and accelerometer and gyroscope for detecting vibrations or obstacles.
- **Authentication and Account Management Service**: Ensures that users can securely register, log in, and manage their personal accounts. The service may rely on internal verification or external APIs.
- **Internet Connectivity**: A stable internet connection is required for the system to upload trip data, publish path information, access external APIs, and display real-time route results.
- **Hosting and Database Infrastructure**: The BBP platform depends on reliable cloud-based infrastructure for persistent storage and retrieval of users' data, paths, and system configurations.

3 Specific requirements

3.1 External Interface Requirements

3.1.1 User Interface

BBP's user interface is implemented as a mobile application suitable for a wide range of modern smartphones. The application must follow the design guidelines for both Android and iOS. The interface should be simple, intuitive, and flexible. This means that the app should adapt to specific user needs, such as adjustable font sizes, light/dark mode, and multiple language options (initially English and Italian, with more planned as the project evolves).

In terms of appearance, the main view of the app consists of a map with a search box for browsing paths, a navigation bar at the bottom, and a login/ registration button in the top-right corner. The navigation bar contains buttons for trip registration, manual path insertion, history, publications, and settings.

Additionally, the application must satisfy the following constraints:

- The app starts in general user mode and allows the user to register to access features available only to registered users.
- The navigation bar is present on all view except the settings page, the login page, and the registration page.
- If the user is not registered or logged in, the navigation bar should be grayed out and inform the user that certain functions are unavailable until the user logs in.
- During trip registration, BBP informs the user about the trip status (active or stopped) using distinct colors.
- The result page (displayed when a trip recording is stopped) shows the measured data.
- The history page provides information on previously biked paths and includes an arrow to access details.
- The confirmation page visualizes the path and lists the detected obstacles.
- The confirmation page allows the user to confirm, update, or reject the path.
- The publication page shows information on the active and archived publications of the user, with a '+' icon to add new entries and a '-' icon to archive them.
- The manual path insertion view allows the user to add streets to the path step by step.

The application uses a unified color palette:

- Light mode: white background with blue and green elements and black text.
- Dark mode: black background with blue and green elements and white text.

3.1.2 Hardware Interfaces

BBP requires a smartphone running iOS or Android. To exchange information (e.g., publish paths, add weather data to trips), the device must support network connectivity. To register trips, it is necessary to obtain live GPS data. More detailed hardware requirements are provided in Chapter [. . .].

3.2 Functional Requirements

Account and Authentication

R1 : The system shall allow a guest user to register by providing email, password, nickname, and optional personal details.

R2 : The system shall send a confirmation email to the user to verify their identity before activating the account.

R3 : The system shall allow a registered user to log in by entering valid credentials.

R4 : The system shall allow a registered user to reset their password via email if forgotten.

R5 : The system shall deny access to users providing invalid credentials.

R6 : The system shall allow registered users to view and update their profile information.

R7 : The system shall allow registered users to delete their account and personal data from the system.

Trip Recording and Statistics

R8 : The system shall allow a registered user to start, pause, and stop the automatic recording of a trip via the mobile interface.

R9 : The system shall continuously collect GPS positions, timestamps, and motion sensor data during an active trip recording.

R10 : The system shall automatically detect if the user is biking based on speed and motion patterns to prevent recording non-cycling activities.

R11 : The system shall retrieve weather data from an external service based on the trip's location and time.

R12 : The system shall store the completed trip, associating it with the user, the calculated statistics, and the weather data.

R13 : The system shall allow registered users to browse their history of recorded trips and view details.

R14 :The system shall allow registered users to publish their recorded trip statistics.

Path Information Contribution

R15 : The system shall allow registered users to create a path entry manually by selecting street segments on a map and specifying the condition (status) and obstacles.

R16 : The system shall allow registered users to edit or delete their own manually inserted path information before it is validated.

R17 : The system shall analyze sensor data during a trip to detect anomalies indicative of potential obstacles.

R18 : The system shall present a summary of the recorded path and detected obstacles to the user at the end of a trip for validation.

R19 : The system shall require the user to confirm, reject, or modify automatically detected obstacles before the data is considered for aggregation.

Path Search and Routing

R20 : The system shall allow any user to search for bike paths by specifying an origin and a destination.

R21 : The system shall compute one or more viable routes connecting the requested origin and destination.

R22 : The system shall rank the computed routes based on a score that combines path condition, safety, distance, and effectiveness.

R23 : The system shall allow users to filter search results based on specific criteria.

R24 : The system shall visualize the selected route on a map, highlighting segments with different statuses.

Data Aggregation and Governance

R25 : The system shall aggregate multiple user contributions referring to the same path segment to compute a unified status.

R26 : The system shall resolve conflicting data by applying the merging policies defined by the Administrator.

R27 : The system shall allow the BBP Administrator to log in to a dedicated management interface.

R28 : The system shall allow the BBP Administrator to configure the parameters for data merging.

R29 : The system shall allow the BBP Administrator to moderate content by removing path information flagged as incorrect, offensive, or spam.

3.2.1 Requirements Mapping

G1: Allow any user with access to the platform to register and create a personal account to access functionalities reserved for registered users.	
Requirements	**Domain Assumptions**
• **R1:** Register as guest user. • **R2:** Email confirmation. • **R3:** Login with credentials. • **R4:** Password reset. • **R5:** Deny invalid access. • **R6:** View/Update profile. • **R7:** Delete account.	• **DA4:** Stable internet connection.

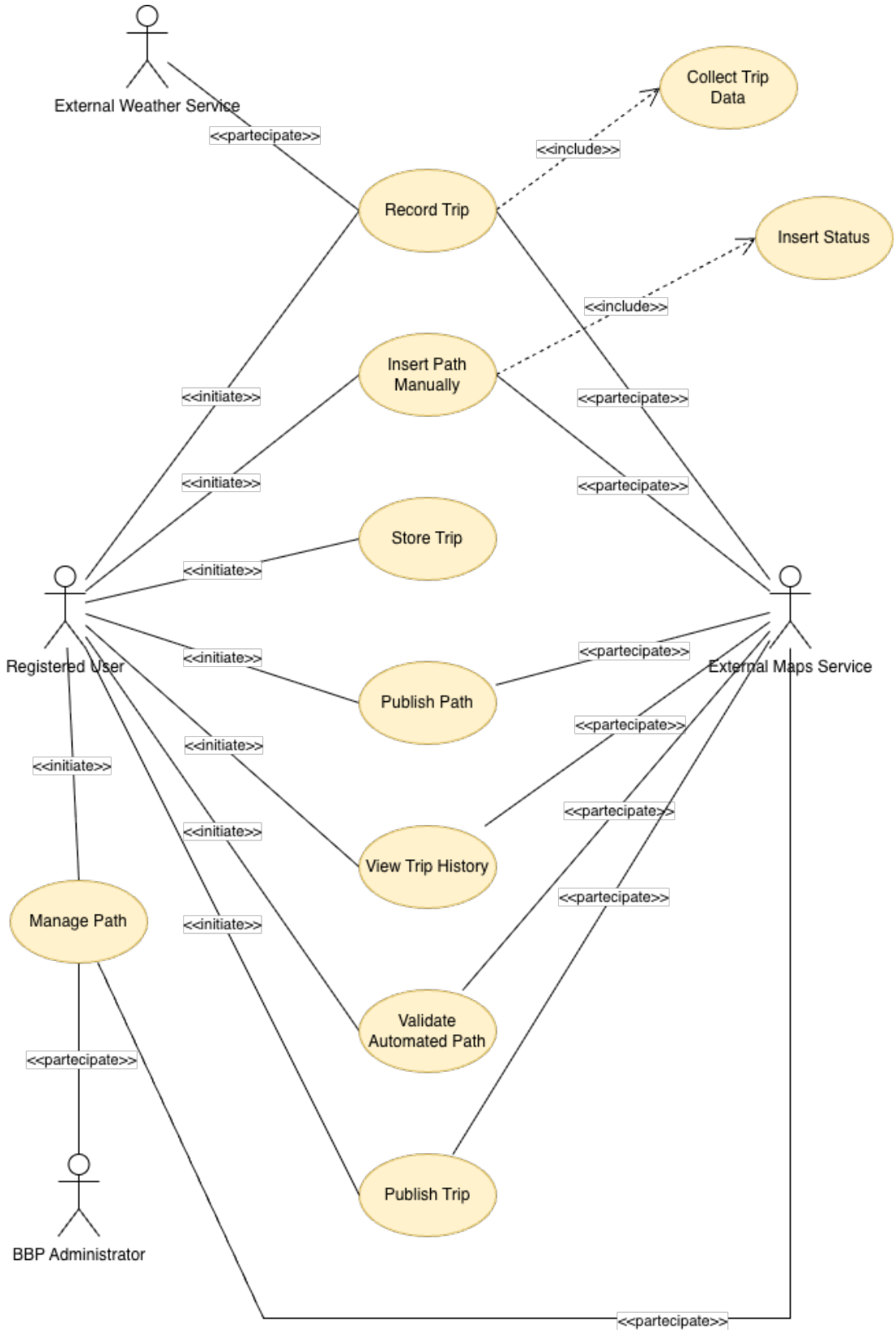
G2: Allow a registered user to record, store and publish the details of their bike trips such as total distance, duration, and average speed enriched, when available, with meteorological information.	
Requirements	**Domain Assumptions**
• **R8:** Start/Stop recording. • **R9:** Collect GPS/Sensor data. • **R10:** Detect biking activity. • **R11:** Retrieve weather data. • **R12:** Store completed trip. • **R13:** Browse trip history. • **R14:** publish trip statistics.	• **DA5:** Device sensors availability. • **DA7:** External weather service. • **DA9:** Sufficient battery life.

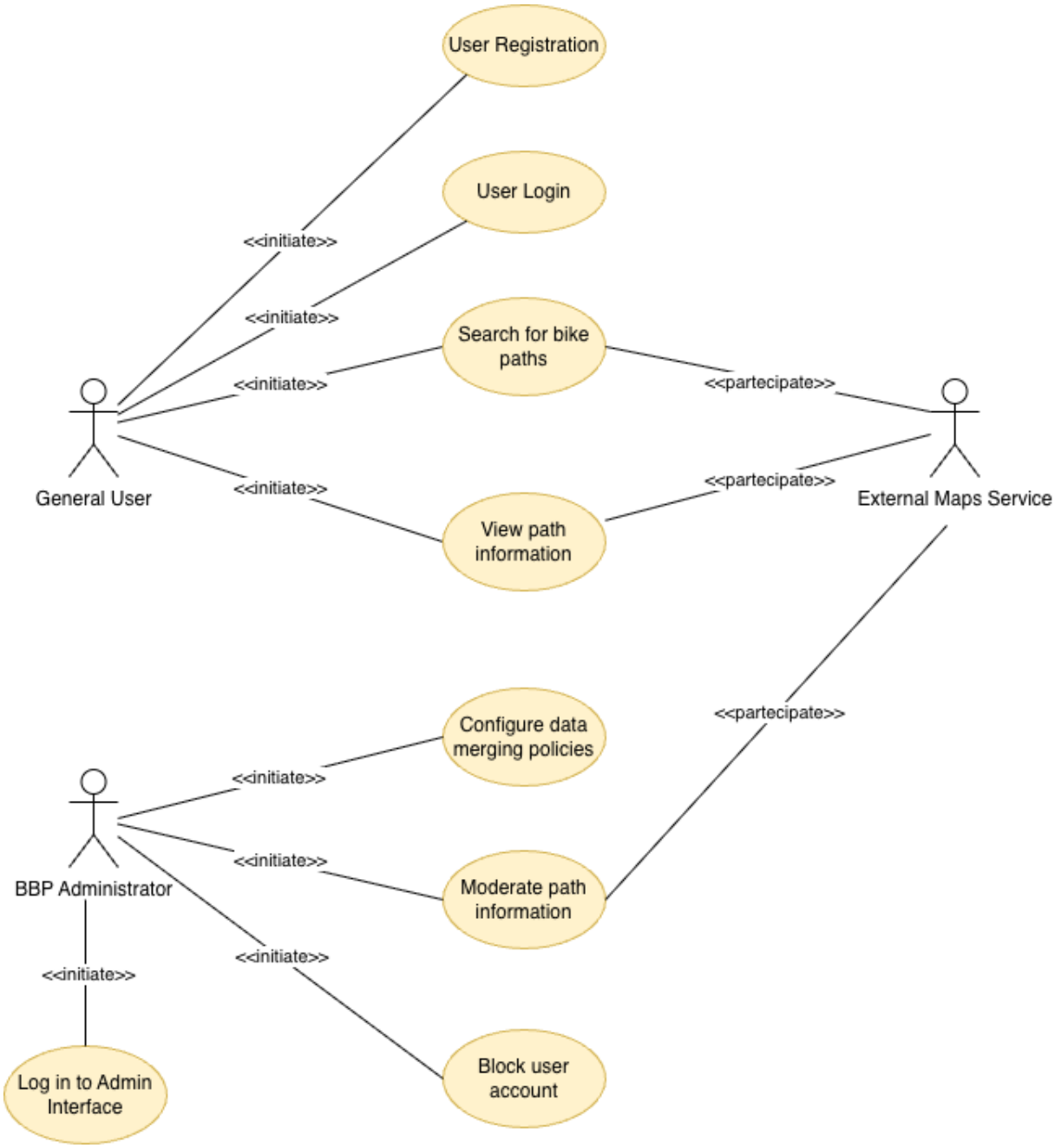
G3: Allow registered users to contribute to updating information about cycling routes by providing data and reports regarding the quality and safety of the paths.	
Requirements	**Domain Assumptions**
• **R15:** Manual path creation. • **R16:** Edit/Delete manual path. • **R17:** Analyze sensor anomalies. • **R18:** Validation summary. • **R19:** Confirm obstacles.	• **DA1:** User manual accuracy. • **DA2:** Validation willingness. • **DA6:** Sensor-Obstacle correlation.

G4: Allow any user to find optimal and relevant bike paths between a specified origin and destination.	
Requirements	**Domain Assumptions**
• **R20:** Search origin/dest. • **R21:** Compute routes. • **R22:** Rank routes by score. • **R23:** Filter results. • **R24:** Visualize route map.	• **DA8:** Map Service accuracy. • **DA4:** Internet connection.

G5: Allow any user to rely on accurate and aggregated information about the condition and quality of bike paths, derived from multiple user contributions.	
Requirements	**Domain Assumptions**
• **R25:** Aggregate contributions. • **R26:** Resolve conflicts. • **R27:** Admin login. • **R28:** Merging policies. • **R29:** Moderate content.	• **DA3:** Admin policy consistency. • **DA10:** Active user community.

3.2.2 Use cases Diagrams




User Registration
User Login
Search for bike paths
View path information
General User
External Maps Service
<<initiate>>
<<initiate>>
<<initiate>>
<<initiate>>
<<partecipate>>
<<partecipate>>
<<partecipate>>
Configure data merging policies
Moderate path information
Block user account
Log in to Admin Interface
BBP Administrator
<<initiate>>
<<initiate>>
<<initiate>>
<<initiate>>

3.2.3 Use cases

UC1 Search for bike paths

Actors	General User, External Maps Service
Entry Condition	• App is installed and opened. • Device is connected with internet network. • User is in *search paths* view.
Event Flow	1. User inserts origin and destination. 2. System validates inserted data. 3. BBP search for possible paths. 4. System visualize possible paths.
Exit Condition	The list of paths is displayed to the user (list can be empty)
Exceptions	• **EX1**: Origin or destination can't be found - app asks user to insert data one more time. • **EX2**: None available paths - app warns user that paths can't be found for specific locations.

UC2 View path information

Actors	General User, External Maps Service
Entry Condition	• Search paths result are viewed. • User has chosen one of the shown path.
Event Flow	1. User clicks on chosen path. 2. BBP views information about path: route on the map, length, expected biking time, status, obstacles. 3. User reviews information as long as needed.
Exit Condition	Path details are shown.
Exceptions	• **EX1**: Path information cannot be shown - system warns user.

UC3 Log in to Admin Interface

Actors	BBP Administrator
Entry Condition	• Administrator is registered in BBP system with administrator's privileges.
Event Flow	1. Administrator opens login panel. 2. Administrator inserts credentials. 3. BBP validates credentials. 4. System accepts or decline administrator.
Exit Condition	Administrator is logged in.
Exceptions	• **EX1**: Incorrect credentials inserted - possible retry.

UC4 Configure data merging policies

Actors	BBP Administrator
Entry Condition	• BBP Administrator is logged in.
Event Flow	1. Administrator opens configuration panel. 2. System shows current merging policy. 3. Administrator changes merging policy. 4. BBP saves new merging policy.
Exit Condition	New merging policies are saved and active.
Exceptions	• **EX1**: parameter value out of range - BBP refuses change.

UC5 Moderate path information

Actors	BBP Administrator
Entry Condition	• BBP Administrator is logged in. • Suspicious path exists.
Event Flow	1. BBP administrator views published paths. 2. Administrator chooses path to moderate. 3. Application shows path details. 4. Administrator edits details. 5. Changes are saved in system.
Exit Condition	Path has been moderated and saved with new data.
Exceptions	None

UC6 Block user account

Actors	BBP Administrator, External Maps Service
Entry Condition	• Administrator is logged into admin panel. • User's account to block exists
Event Flow	1. Administrator reviews list of users in administrator panel. 2. Administrator picks user to block. 3. Administrator chooses option *Block Account*. 4. BBP views confirmation box. 5. Administrator confirms operation. 6. User's account is marked as blocked.
Exit Condition	User's account is blocked and user cannot log in and use `RegisteredUser` features
Exceptions	• **EX1**: Account is already blocked - BBP informs administrator about that.

UC7 User Registration

Actors	General User
Entry Condition	• User is not already registered.
Event Flow	1. User selects the option to create a new account. 2. User enters registration details (email, password, nickname). 3. BBP validates the input format and uniqueness. 4. BBP sends a confirmation email. 5. User clicks the confirmation link in the email. 6. BBP activates the user account.
Exit Condition	User account is created and activated.
Exceptions	• **EX1**: Email already in use - BBP informs user and requests a different. email

UC8 User Login

Actors	Registered User
Entry Condition	• The User has to be registered.
Event Flow	1. User enters email and password. 2. BBP verifies credentials. 3. BBP grants access and redirects user to the dashboard.
Exit Condition	User is authenticated and can access all personal and community features.
Exceptions	• **EX1**: Invalid credentials - BBP displays an error message and allows retry. • **EX2**: Account blocked - BBP informs user to contact support.

UC9 Record Trip

Actors	Registered User, External Weather Service, External Maps Service
Entry Condition	• The user is logged into the system. • The device GPS and motion sensors are active and functioning.
Event Flow	1. The user initiates the "Record Trip" function from the main interface. 2. The system continuously retrieves GPS coordinates and sensor data (accelerometer/gyroscope). 3. The system queries the External Maps Service to visualize the current position on the map. 4. The system queries the External Weather Service to attach current meteorological metadata. 5. The system automatically detects cycling activity (based on speed). 6. The user manually stops the recording.
Exit Condition	Trip data (path, statistics, detected obstacles, weather) is collected in temporary storage, awaiting saving.
Exceptions	• **EX1: GPS Signal Lost:** The system pauses the recording and notifies the user. • **EX2: Weather Service Unavailable:** The trip is recorded without weather data (or with partial data).

UC10 Insert Path Manually

Actors	Registered User, External Maps Service
Entry Condition	• The user is logged in. • Internet connection is available to load the map.
Event Flow	1. The user selects "Insert Path Manually". 2. The system displays the map via the External Maps Service. 3. The user selects start and end points or draws the route on the map. 4. The user inserts the path status and adds any obstacles. 5. The user saves the path.
Exit Condition	A new path entry is created in the user's private collection.
Exceptions	• **EX1: Map Service Unavailable:** The map fails to load; the user cannot select the route.

UC11 Store Trip

Actors	Registered User
Entry Condition	• A trip recording has just been stopped or completed.
Event Flow	1. The system displays the trip summary (distance, duration, average speed, map). 2. The user reviews the summary. 3. The user selects the "Save" option. 4. The system persists the trip data to the database linked to the user account.
Exit Condition	The trip is permanently saved in the user's personal history.
Exceptions	• **EX1: Storage Error:** The system fails to save data (memory full or DB error) and prompts the user to retry.

UC12 Publish Path

Actors	Registered User
Entry Condition	• Registered User is logged in. • At least one path owned by the user exists and is validated.
Event Flow	1. Registered User opens the list of own paths. 2. System displays all paths with visibility status. 3. Registered User selects a path eligible for publication. 4. System shows the path details. 5. Registered User chooses "Publish path". 6. System checks eligibility and marks the path as public. 7. System confirms successful publication.
Exit Condition	The selected path is stored as public and visible to other users.
Exceptions	• **EX1**: Path is not eligible for publication. • **EX2**: Path is already public.

UC13 View Trip History

Actors	Registered User
Entry Condition	• Registered User is logged in.
Event Flow	1. Registered User selects "Trip history". 2. System retrieves all trips owned by the user. 3. System displays the list of trips. 4. Registered User selects one trip from the list. 5. System shows detailed trip information.
Exit Condition	Trip history (and optionally trip details) is displayed to the user.
Exceptions	• **EX1**: No trips found for the user.

UC14 Validate Automated Path

Actors	Registered User
Entry Condition	• Registered User is logged in. • At least one automatically generated path is stored as unvalidated.
Event Flow	1. Registered User opens the list of unvalidated paths. 2. System displays the unvalidated paths. 3. Registered User selects a path to validate. 4. System shows the reconstructed path and detected obstacles. 5. Registered User confirms, edits, or removes obstacles. 6. Registered User confirms validation of the path. 7. System stores changes and marks the path as validated.
Exit Condition	The selected path is validated and ready for publication or merging.
Exceptions	• **EX1**: User rejects the path entirely.

UC15 Publish Trip

Actors	Registered User
Entry Condition	• Registered User is logged in. • At least one stored trip exists.
Event Flow	1. Registered User opens “Trip history”. 2. System shows list of stored trips. 3. Registered User selects a trip. 4. System shows trip details and publication state. 5. Registered User selects “Publish trip”. 6. System validates publishability. 7. System marks the trip as public and confirms the update.
Exit Condition	The selected trip statistics are published and visible to other users.
Exceptions	• **EX1**: Trip already published.

UC16 Manage Path

Actors	Registered User, BBP Administrator
Entry Condition	• Actor is logged in. • At least one path exists that the actor is allowed to manage.

Event Flow	1. Actor opens the "Manage paths" section.
	2. System displays manageable paths.
	3. Actor selects a path.
	4. System shows complete path details.
	5. Actor selects a management action:
	  • Delete path: action reserved to the administrator
	  • edit obstacles: action to be confirmed by the administrator
	6. System checks permissions and applies the requested change.
	7. Administrator confirms the update
	8. The system shows the modified path.

Exit Condition	Requested change is correctly applied to the selected path.
Exceptions	• **EX1**: Administrator rejects the change (if moderation is required).

3.2.4 Use Cases' Sequence Diagrams

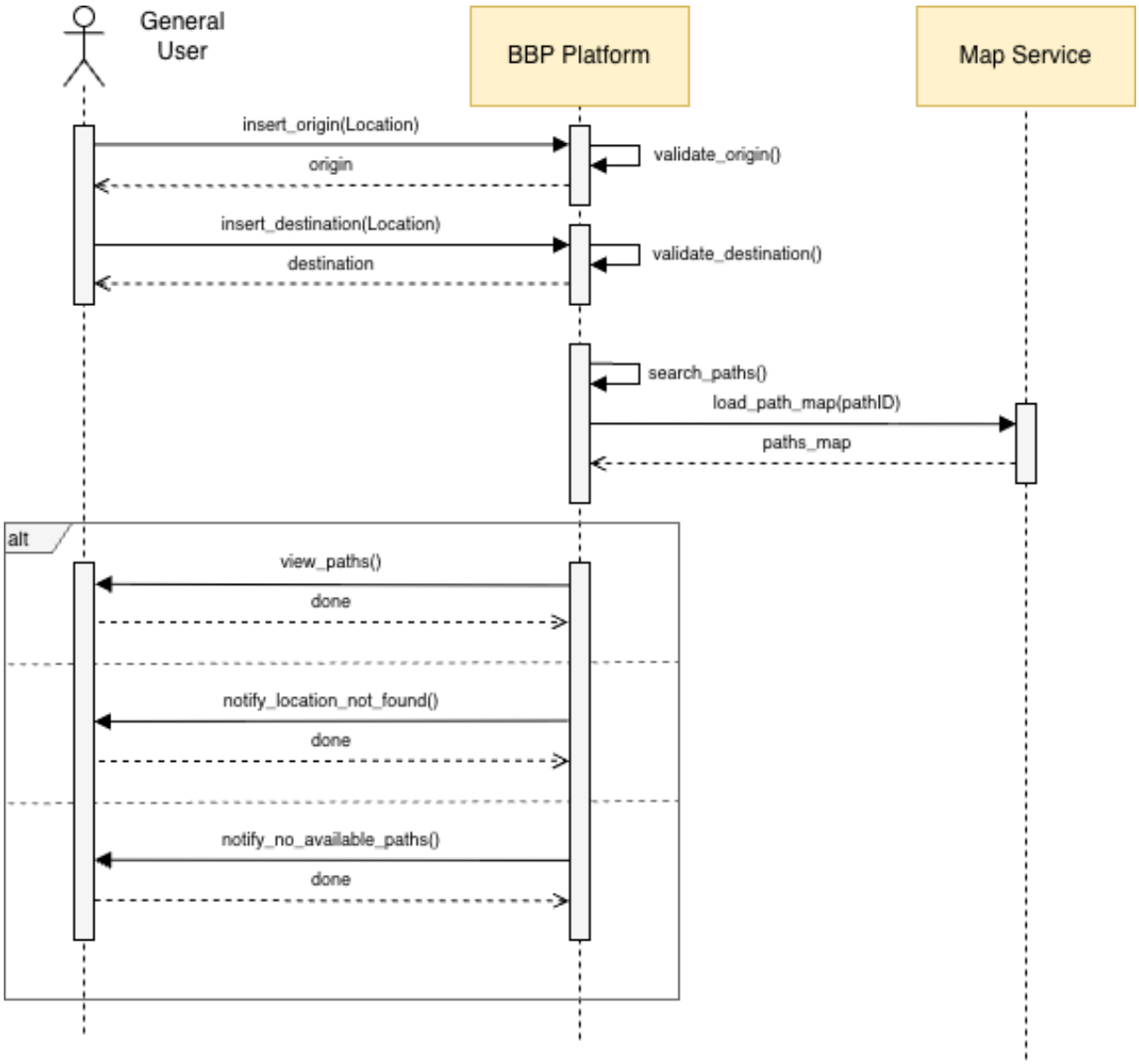


Figure 1: UC1 Search for bike paths

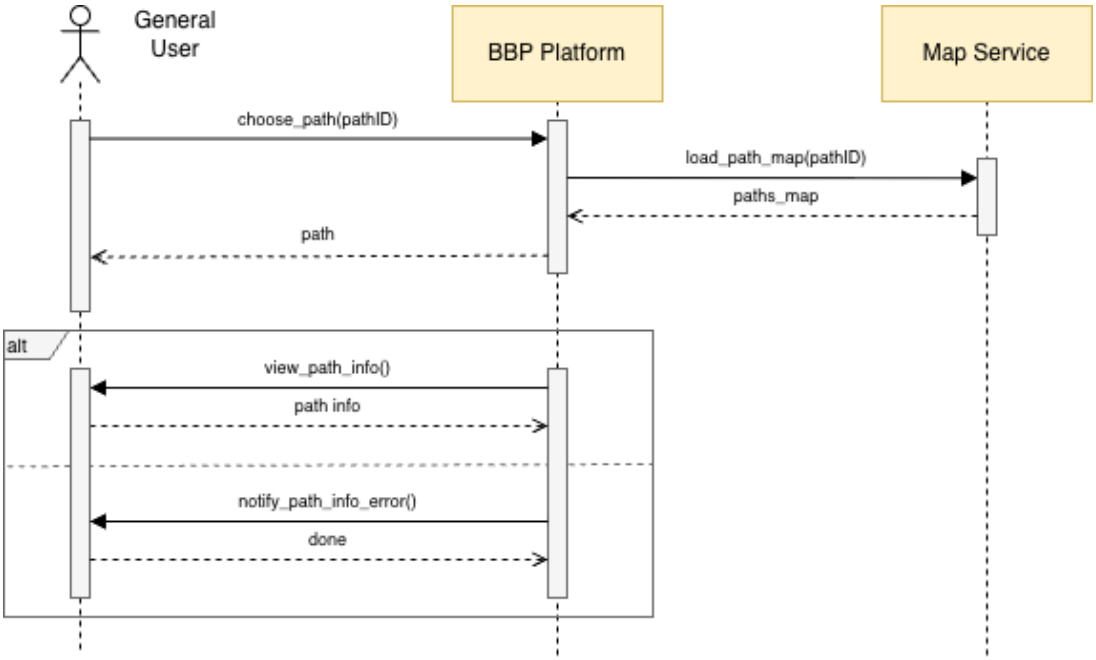


Figure 2: UC2 View path information

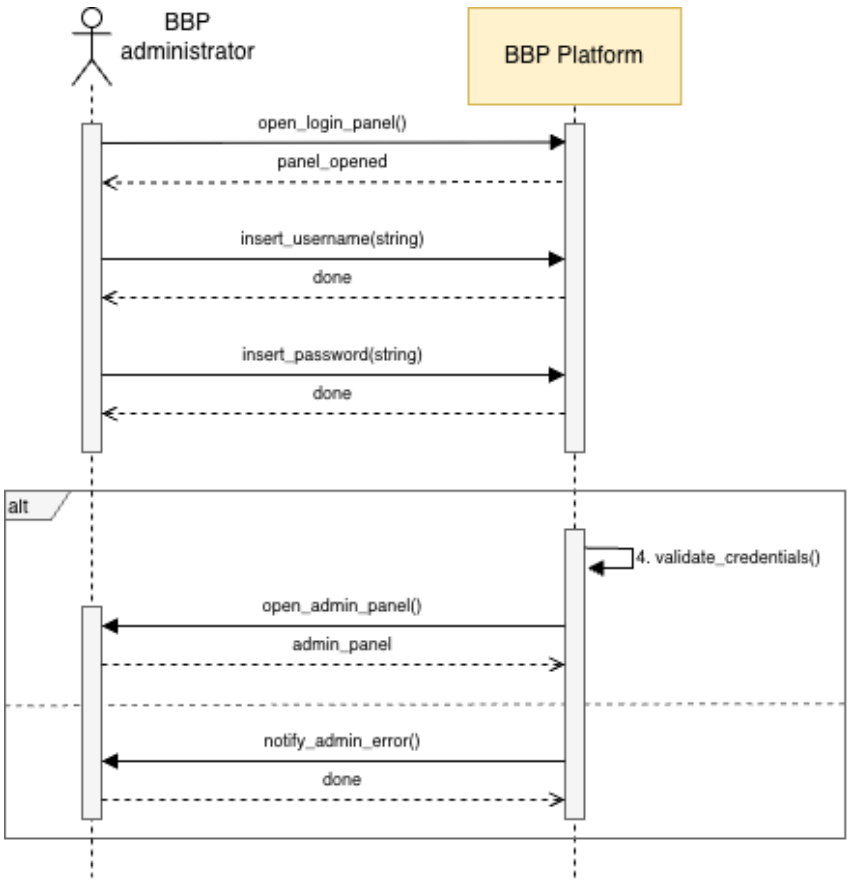


Figure 3: UC3 Log in to Admin Interface

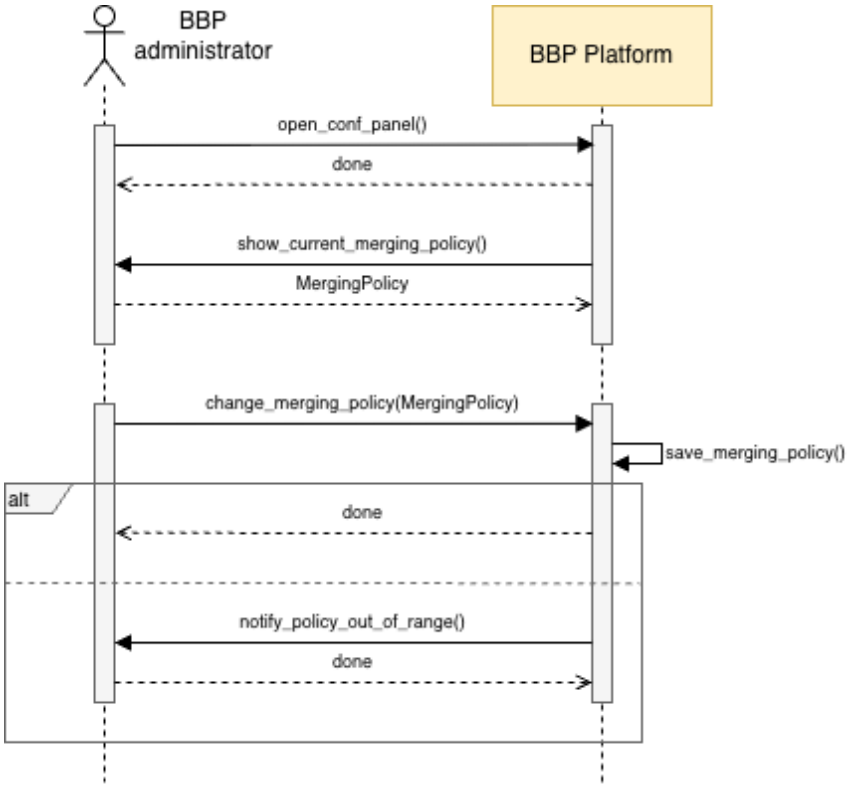


Figure 4: UC4 Configure data merging policies

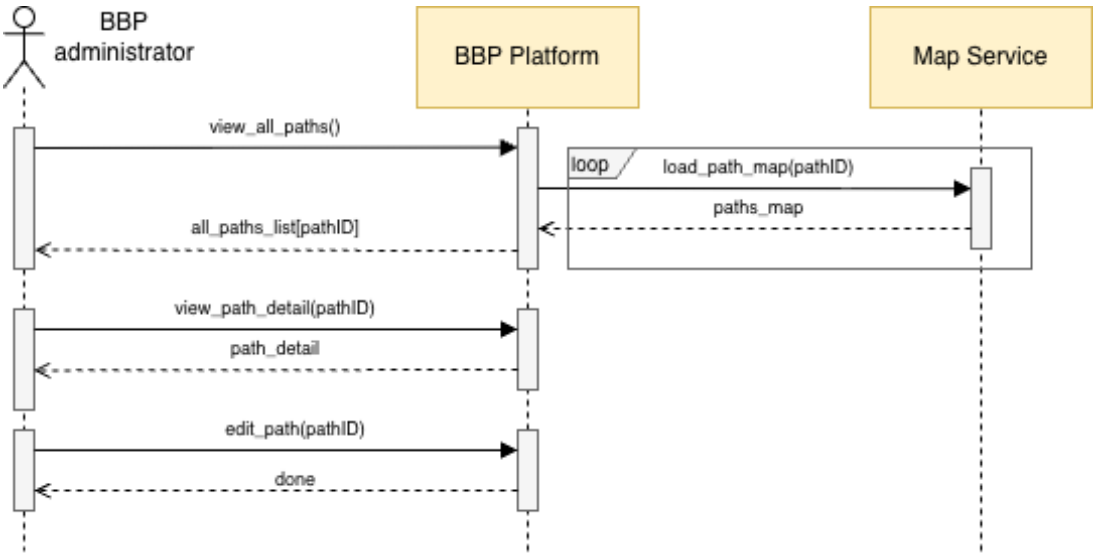


Figure 5: UC5 Moderate path information

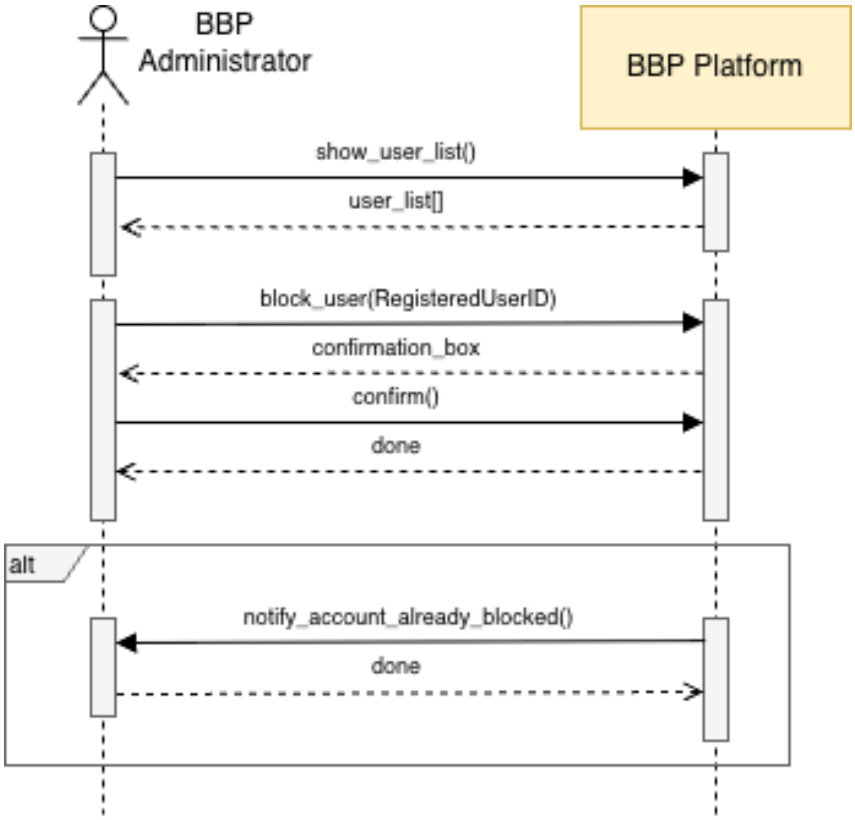


Figure 6: UC6 Block user account

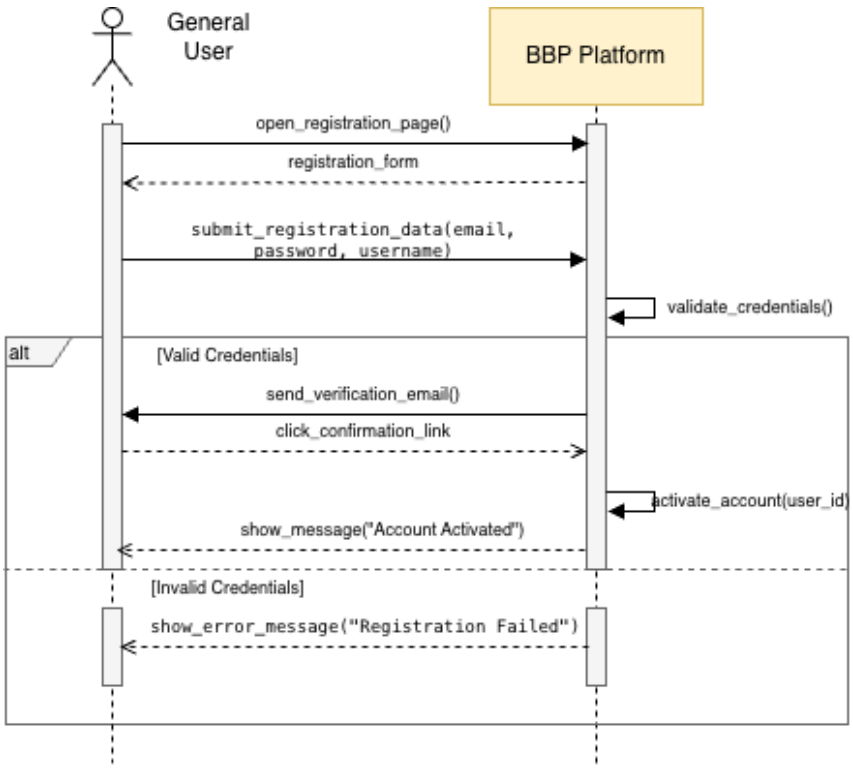


Figure 7: UC7 User Registration

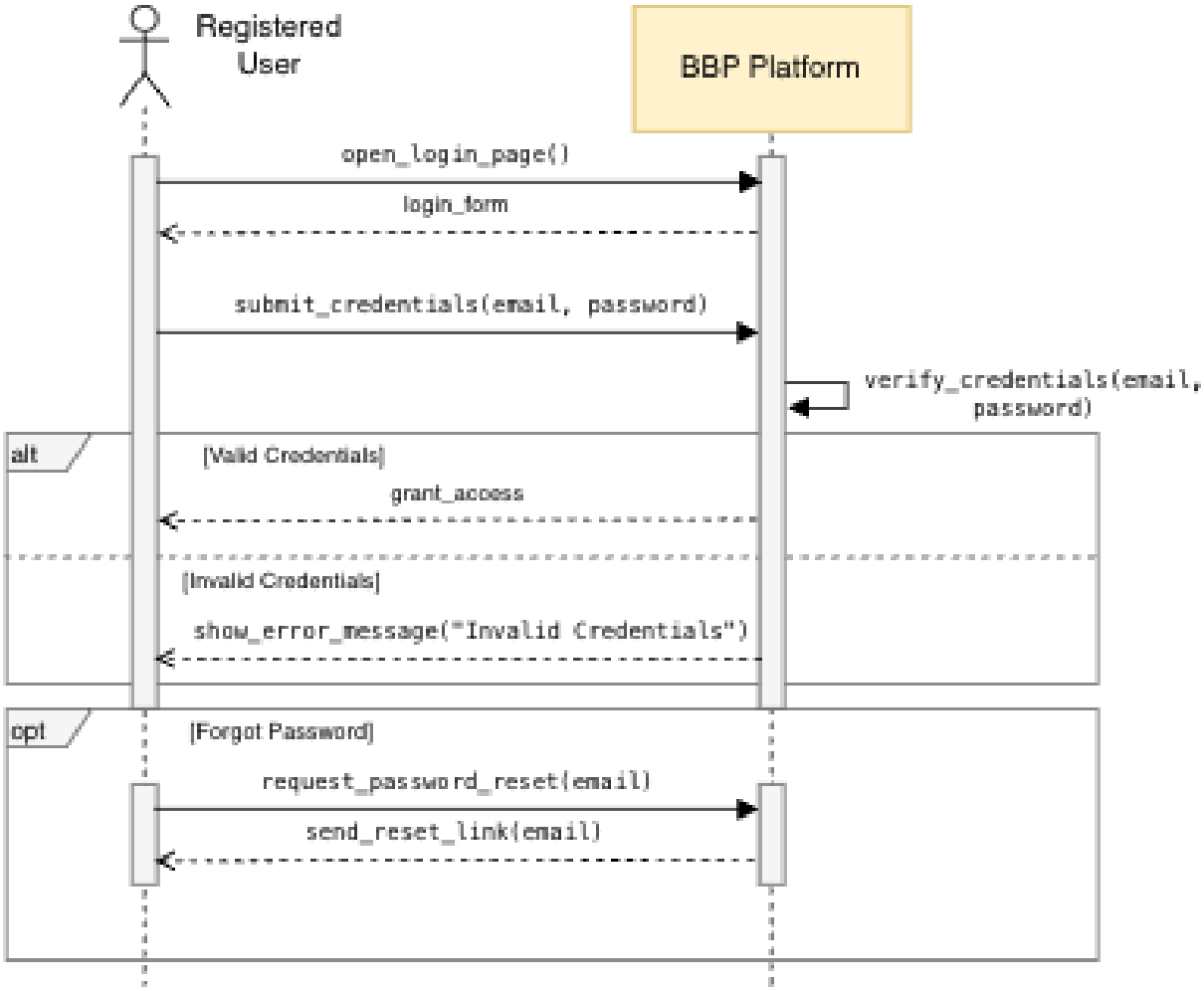


Figure 8: UC8 User Login

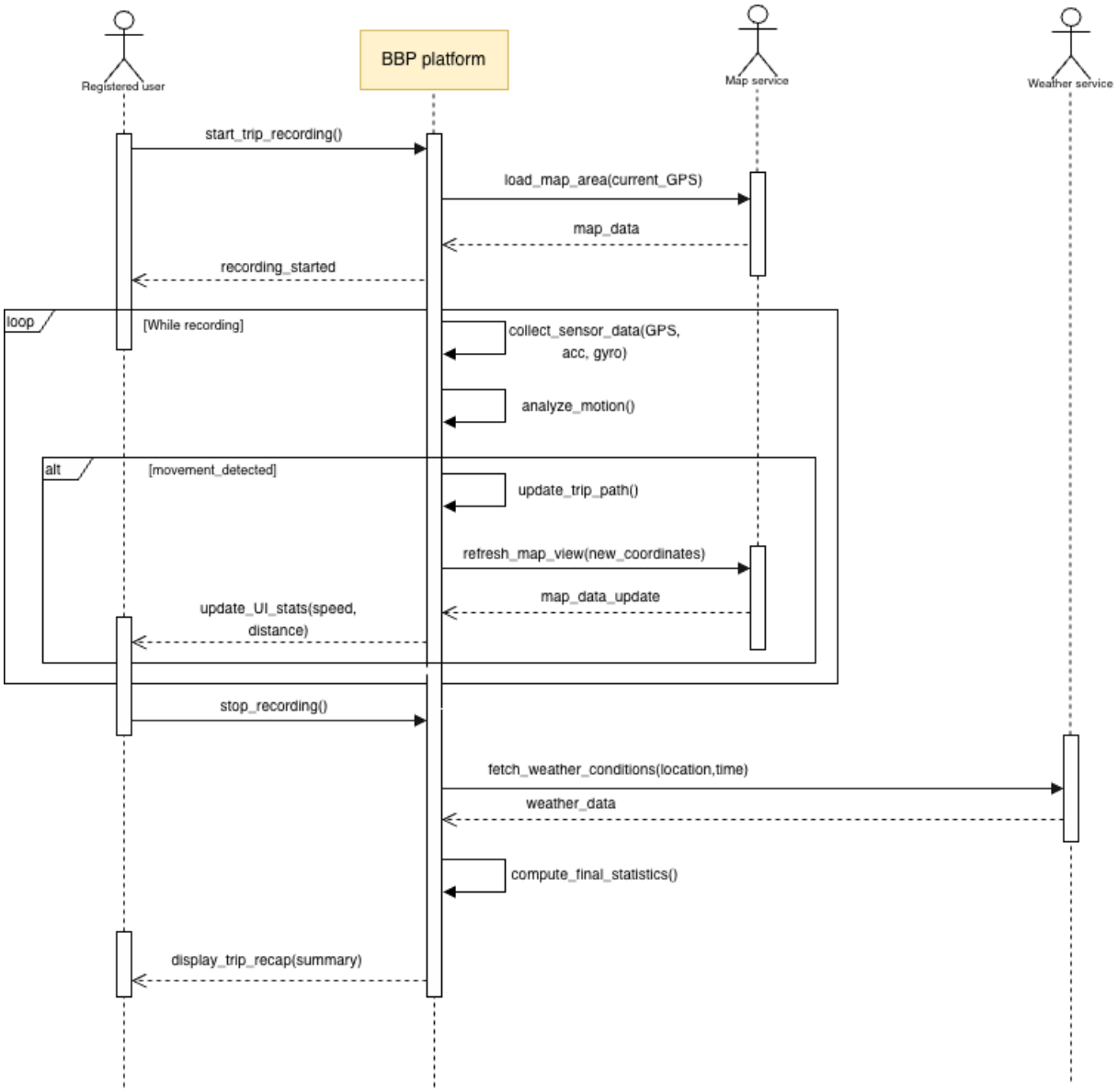


Figure 9: UC9 Record Trip

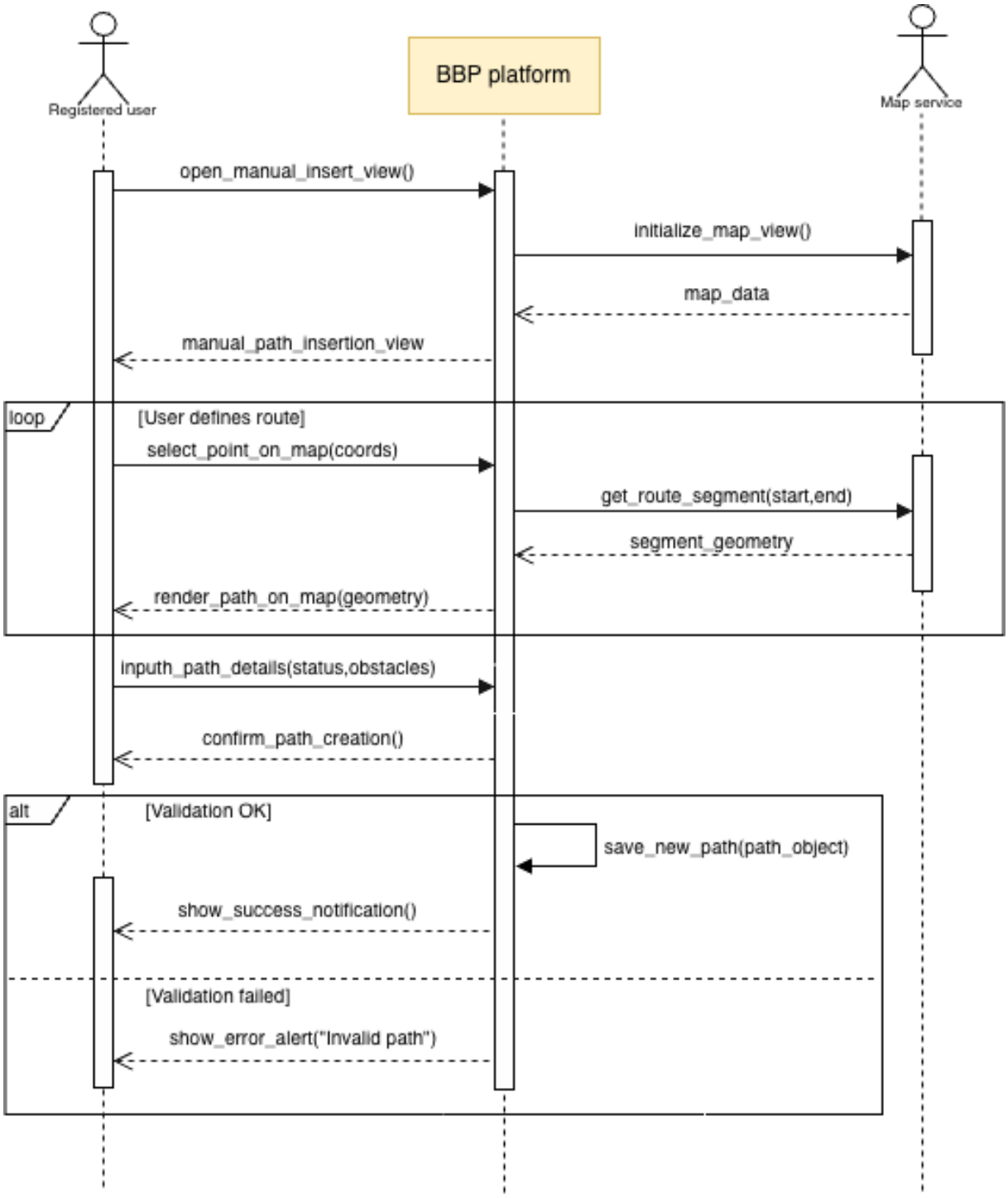


Figure 10: UC10 Insert Path Manually

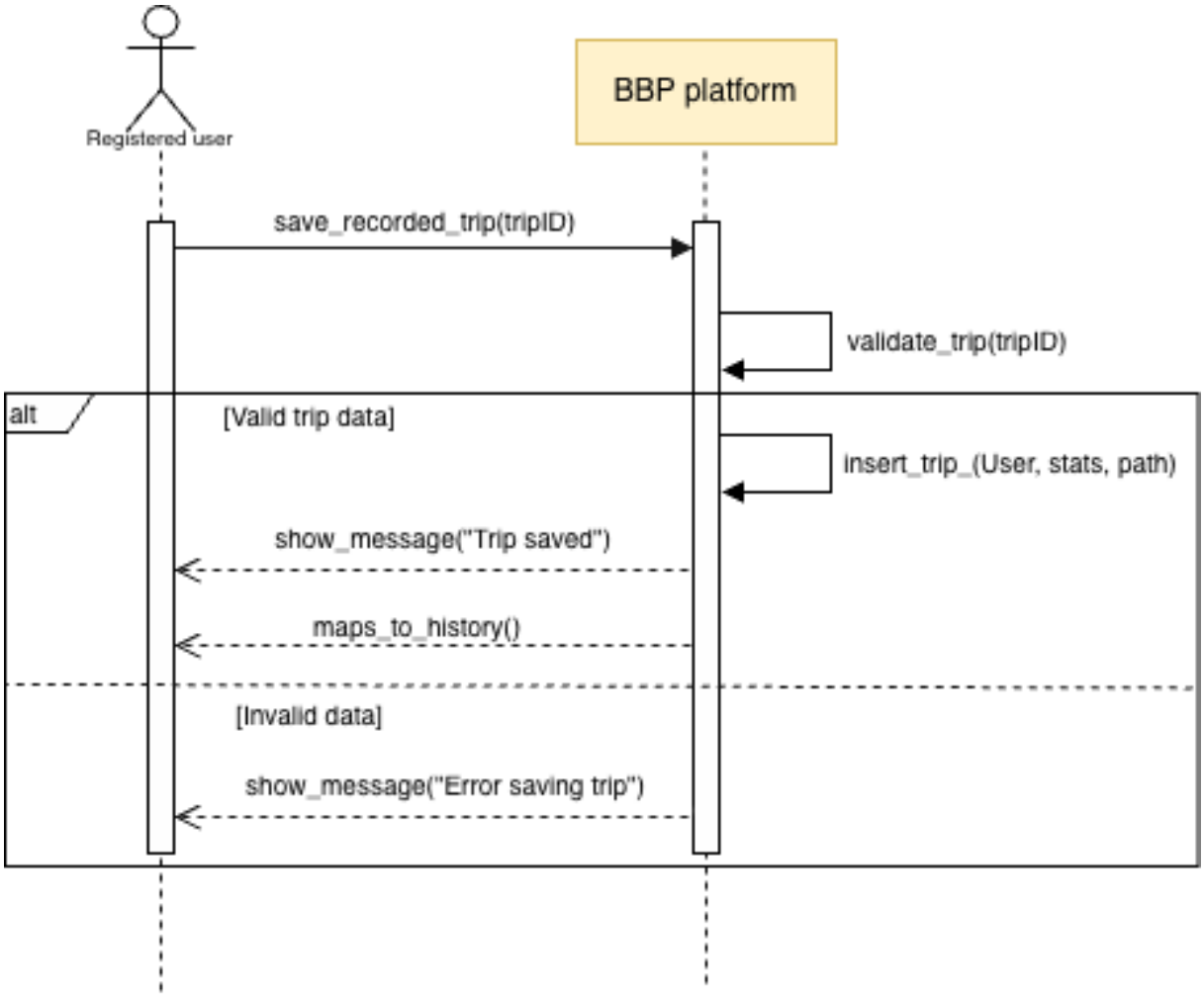


Figure 11: UC11 Store Trip

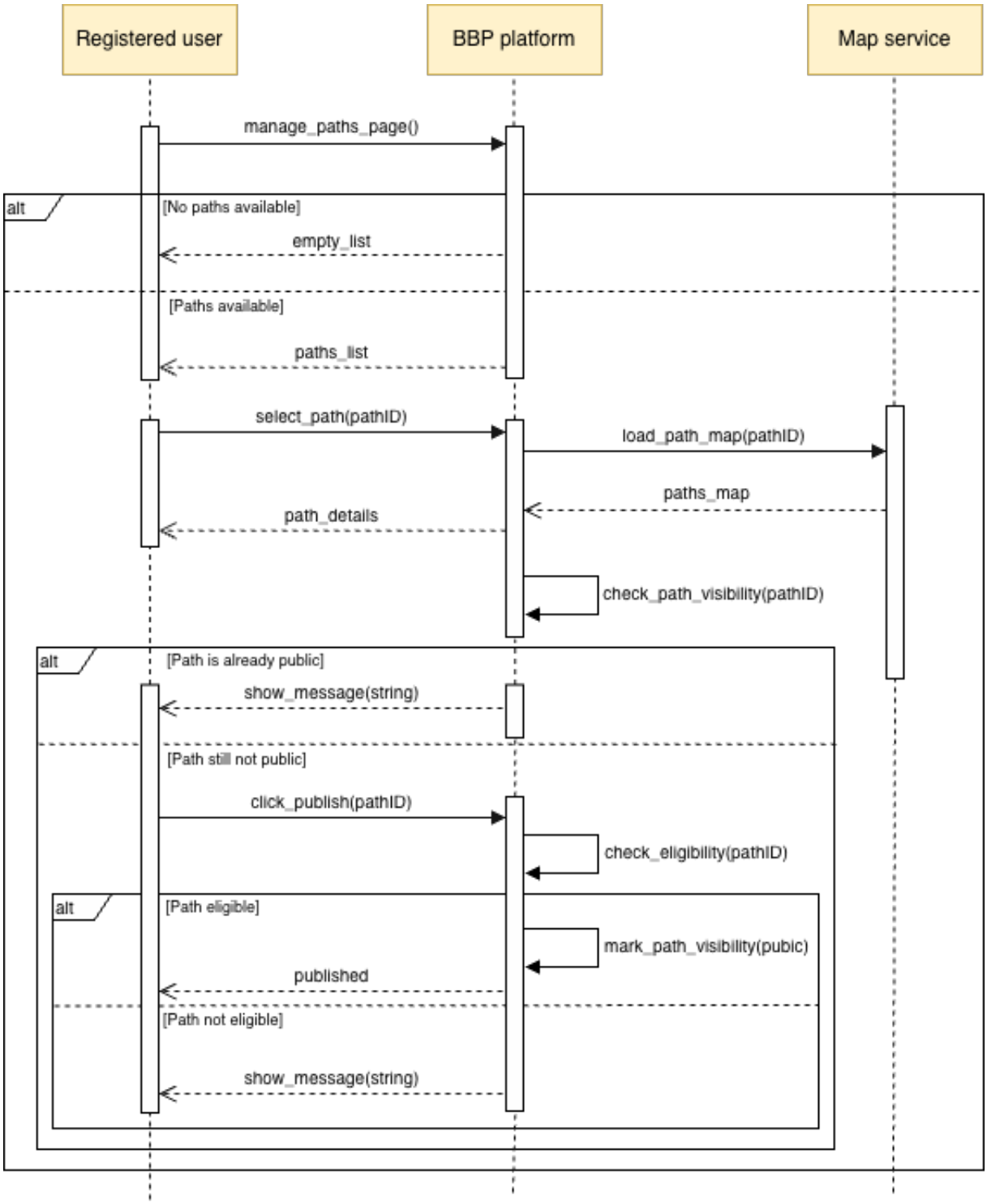


Figure 12: UC12 Publish Path

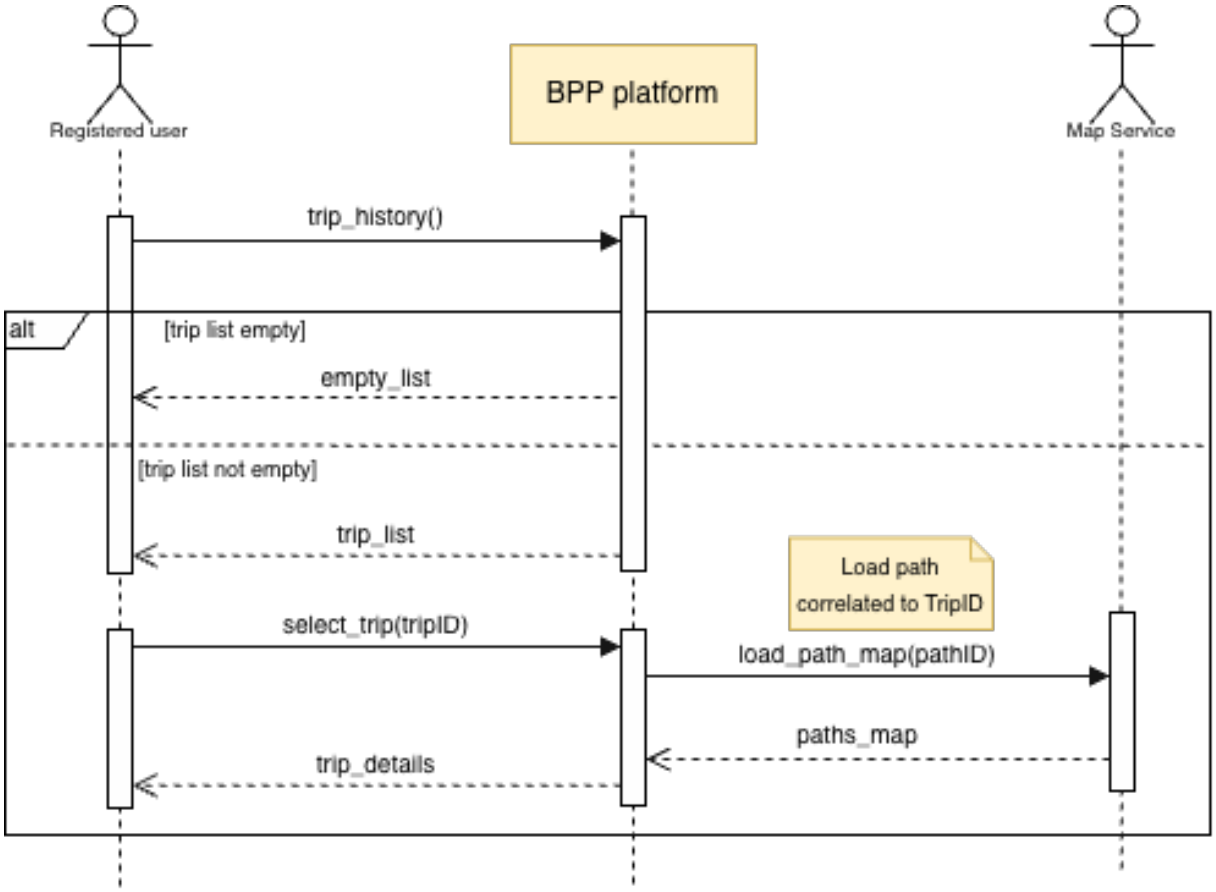
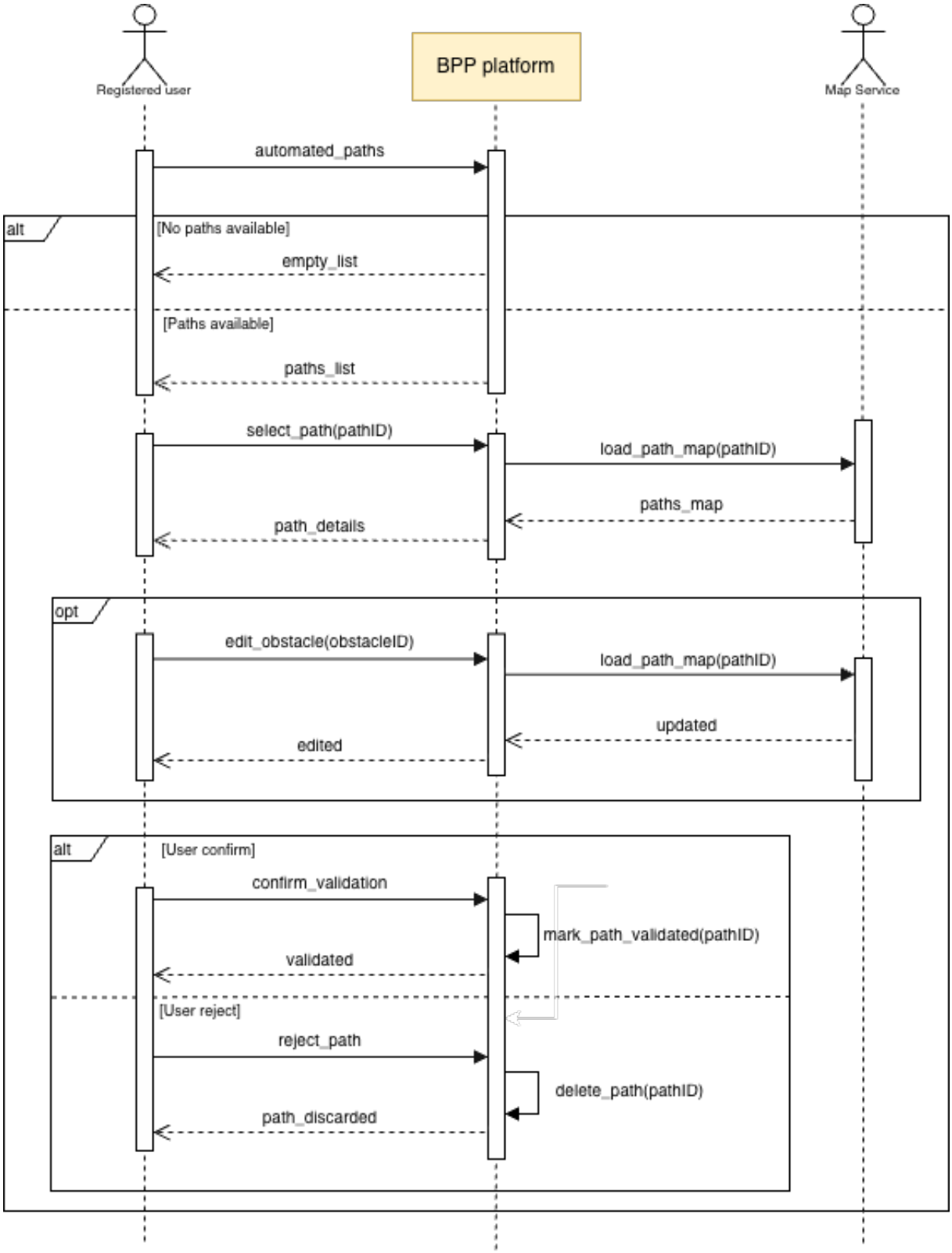


Figure 13: UC13 View Trip History


Registered user
BPP platform
Map Service
automated_paths
alt
[No paths available]
empty_list
[Paths available]
paths_list
select_path(pathID)
load_path_map(pathID)
paths_map
path_details
opt
edit_obstacle(obstacleID)
load_path_map(pathID)
updated
edited
alt
[User confirm]
confirm_validation
mark_path_validated(pathID)
validated
[User reject]
reject_path
delete_path(pathID)
path_discarded

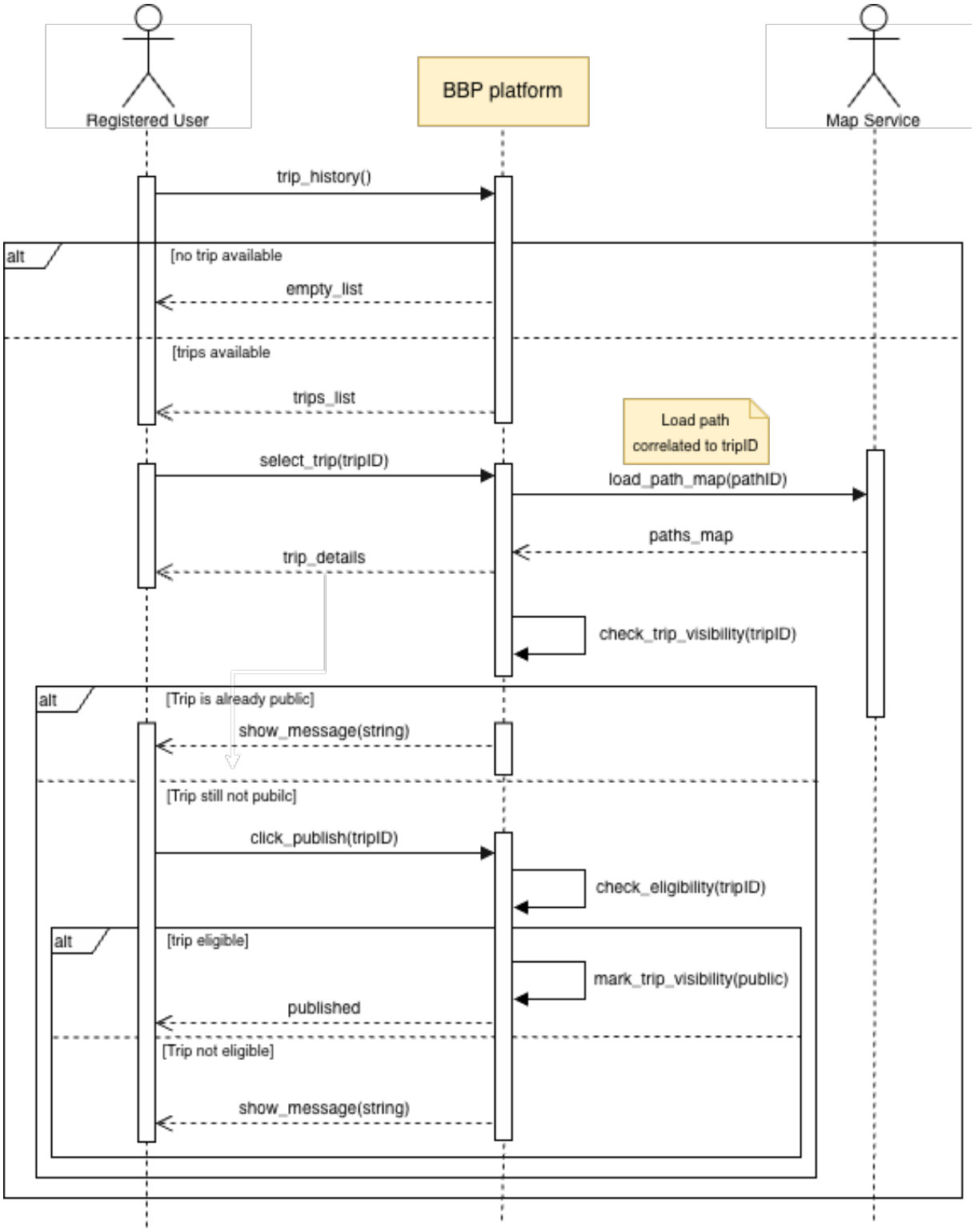
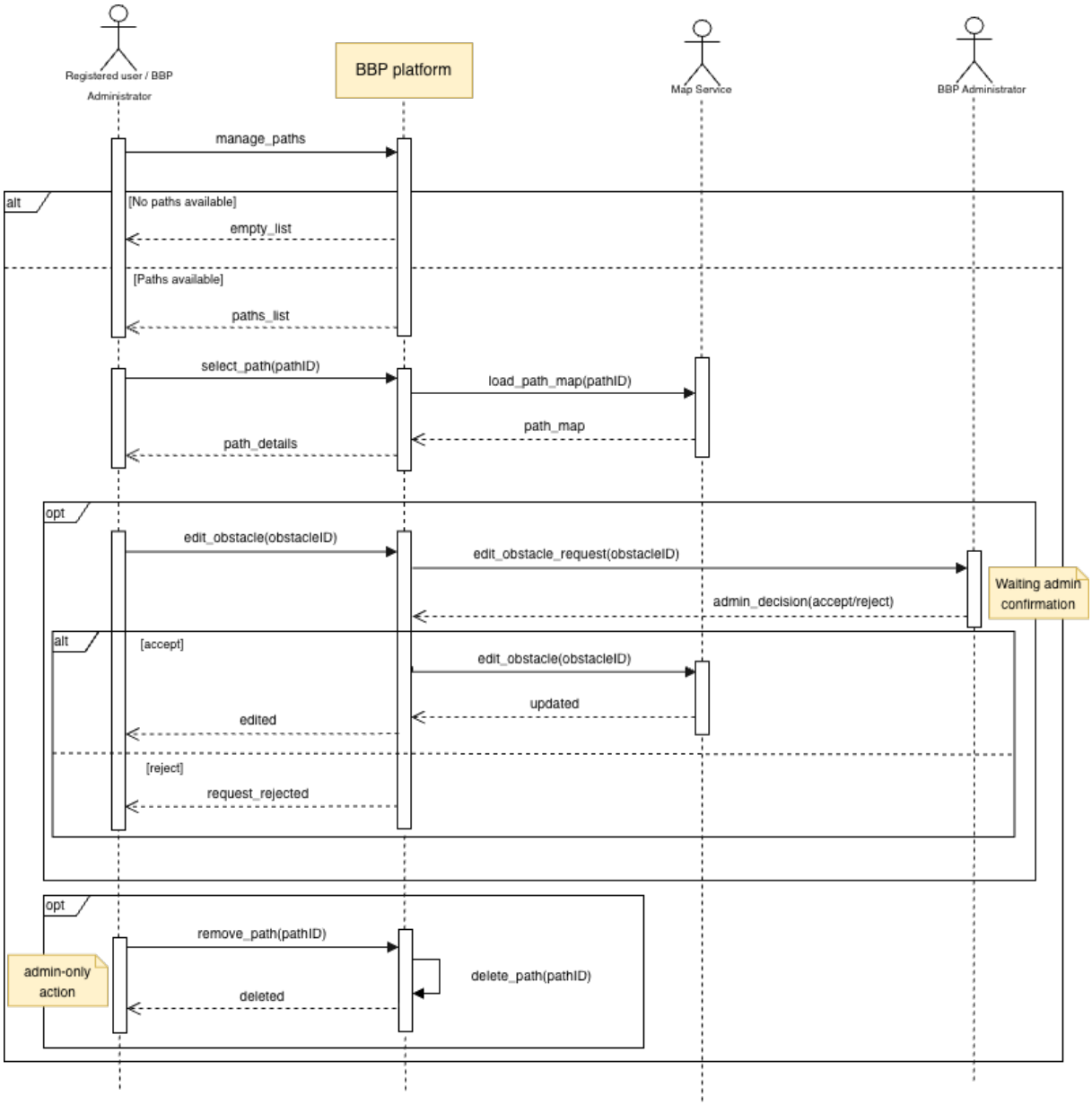


Figure 15: UC15 Publish Trip


Registered user / BBP Administrator
BBP platform
Map Service
BBP Administrator
manage_paths
alt
[No paths available]
empty_list
[Paths available]
paths_list
select_path(pathID)
load_path_map(pathID)
path_map
path_details
opt
edit_obstacle(obstacleID)
edit_obstacle_request(obstacleID)
Waiting admin confirmation
admin_decision(accept/reject)
alt
[accept]
edit_obstacle(obstacleID)
updated
edited
[reject]
request_rejected
opt
remove_path(pathID)
admin-only action
delete_path(pathID)
deleted

3.3 Performance Requirements

System Scalability and User Load
BBP needs to be developed to accommodate an expanding community of cyclists and casual users who utilize the platform to explore routes or check trail conditions. Consequently, the system must support thousands of concurrent users without any significant delays. Specifically, BBP must ensure consistent performance even when numerous users are logging trips simultaneously or when multiple requests for route calculations are made at the same time. There should be no rigid maximum placed on the number of accounts registered or the overall quantity of trip data gathered.

Real-Time Data Acquisition Capabilities
Given that BBP depends on real-time data gathered from GPS, accelerometers, and gyroscopes while automatically recording trips, the system needs to be able to handle sensor inputs constantly and with low latency. Data streams need to be collected without loss, buffering problems, or disruptions, guaranteeing an accurate restoration of the route taken. The system should function effectively on mobile devices, minimizing significant delays or processing burdens while tracking trips.

Aggregated Data Processing and Availability
As BBP continuously receives new path reports and updated information from the community, the system must merge these contributions efficiently so that the resulting path conditions remain consistently up-to-date.

3.4 Design Constraints

3.4.1 Standards compliance

- **GDPR Compliance**

  BBP must process personal and location data in accordance with GDPR principles. Only information strictly necessary for user identification, trip storage, and path quality reporting may be collected. All data must be handled in a way that prevents any form of re-identification when presented in aggregated or statistical form.

- **Data Protection and Security**

  The system must adopt established information security practices to ensure confidentiality, integrity, and controlled access to sensitive data such as GPS traces and sensor-based obstacle reports. Communication between client and server must occur over secure channels.

3.4.2 Hardware Limitations

- **Sensor Requirements:** To enable automatic trip recording and surface-issue detection, devices accessing BBP must provide functional GPS, accelerometer, and gyroscope sensors. Without these components, only manual input features can be used.
- **Device Capabilities:** The application must operate on devices with sufficient memory and processing capacity to handle continuous sensor acquisition and background trip reconstruction. A stable internet connection is required for map loading, data synchronization, and retrieval of meteorological information.
- **User Permissions:** Users must explicitly grant permissions for location services and motion sensors; without them, the system cannot perform automatic tracking.
- **Compatibility Expectation:** BBP should remain accessible on the vast majority of modern smartphones and browsers, ensuring broad usability without requiring high-end hardware.

3.5 Software System Attributes

3.5.1 Reliability

The system must maintain a high level of reliability by ensuring continuous operation even in the presence of failures in data centers, communication networks, or other components of the infrastructure. All critical data required for system functionality should be redundantly stored and replicated in multiple independent locations to prevent data loss and minimize service interruptions. Fault-tolerant mechanisms, such as automatic failover, load balancing, and recovery procedures, should be implemented to ensure that the system remains operational and responsive despite hardware or network disruptions.

3.5.2 Availability

The BBP system should maintain a high level of availability, with a guaranteed minimum uptime of 99.9%. Under this requirement, the system may be unavailable (e.g. due to maintenance) for a maximum of approximately 9 hours per year. Any planned downtime should be communicated to users several days in advance.

3.5.3 Security

As the system holds confidential information about registered users, it must always establish communication using industry-standard security protocols such as HTTPS. User credentials should be stored using strong hashing algorithms. In addition, the system must be protected against common threats, such as SQL injection, cross-site scripting, and brute-force attacks.

3.5.4 Maintainability

The system should be implemented in a way that is easy to scale, maintain, and test. The code must remain readable, maintainable, and fully documented. Maintenance breaks should be scheduled during the night or during other low-use periods. Major or disruptive updates should be tested during the winter season, when user activity is significantly lower.

3.5.5 Portability

The system should be accessible from most Android and iOS devices. The application must be optimized to run reliably on a wide range of screen sizes, performance levels, and device capabilities.

4 Formal analysis using alloy

4.1 Alloy model

This chapter presents a formal model of selected aspects of the *Best Bike Paths* (BBP) system using the Alloy specification language. The model focuses on the core concepts introduced in the previous sections of this RASD, with the goal of capturing their structure and constraints in a precise and analyzable way.

The Alloy specification reflects the main entities of the BBP domain, such as users and administrators, paths and trips, obstacles, and the mechanisms used to evaluate, publish, and moderate path information. Particular attention is given to the relationships between these entities, including ownership rules, lifecycle states of paths, aggregation of community feedback, and administrative governance actions.

```
-----------------------------------------------------------------
-- IDs and enums
-----------------------------------------------------------------

sig UID {}
sig PathID {}
sig TripID {}
sig ObstacleID {}
sig StatusID {}
sig ReportID {}

enum Visibility { Public, Private }
enum ReviewState { Unconfirmed, Confirmed, Rejected }
enum PublicationState { Unpublished, Published, Archived }

enum ObstacleKind { Pothole, RoadWorks, FallenTree, Other }
enum ObstacleDecision { Pending, ConfirmedObs, RejectedObs }

enum PathCondition { Good, Caution, Bad }

-----------------------------------------------------------------
-- Users
-----------------------------------------------------------------

abstract sig User { uid: one UID }

sig Guest extends User {}              -- unregistered user
sig RegisteredUser extends User {}     -- registered user
sig Admin extends RegisteredUser {}    -- BBP administrator

fact UniqueUID {
```

```
  all disj u1, u2: User | u1.uid != u2.uid
}

fact AtLeastOneAdmin { some Admin }

-----------------------------------------------------------------
-- Trips and Paths
-----------------------------------------------------------------

sig Trip {
  tid: one TripID,
  rider: one RegisteredUser,

  pathRef: one Path
}

abstract sig Path {
  pid: one PathID,
  owner: one RegisteredUser,

  review: one ReviewState,
  visibility: one Visibility,
  publication: one PublicationState
}

sig ManualPath extends Path {}
sig AutoPath extends Path {
  derivedFrom: one Trip
}

fact UniqueIDs {
  all disj p1, p2: Path | p1.pid != p2.pid
  all disj t1, t2: Trip | t1.tid != t2.tid
}

fact TripPathRefCoherence {
  -- Trip references a path owned by the same registered user
  all t: Trip | t.rider = t.pathRef.owner
}

fact AutoPathConsistency {
  -- Automatic paths are derived from a trip of the same user
  all p: AutoPath | p.owner = p.derivedFrom.rider
}

fact AutoPathTripLink {
  -- every AutoPath is exactly the path referenced by its derived trip
  all p: AutoPath | p.derivedFrom.pathRef = p
```

```
  -- if a trip references an AutoPath, that AutoPath must be derived
     from the trip
  all t: Trip | (t.pathRef in AutoPath) implies (t.pathRef.derivedFrom
     = t)
}

fun userTrips[u: RegisteredUser] : set Trip { { t: Trip | t.rider = u }
   }
fun ownedPaths[u: RegisteredUser] : set Path { { p: Path | p.owner = u
   } }

-----------------------------------------------------------------
-- Obstacles
-----------------------------------------------------------------

sig Obstacle {
  oid: one ObstacleID,
  kind: one ObstacleKind,
  onPath: one Path,

  -- merged decision
  decision: one ObstacleDecision,
  decidedBy: lone RegisteredUser,

  reports: set ObstacleReport
}

sig ObstacleReport {
  rid: one ReportID,
  reporter: one RegisteredUser,
  obstacle: one Obstacle,
  reportedKind: one ObstacleKind,
  reportDecision: one ObstacleDecision
}

fact UniqueObstacleIDs {
  all disj o1, o2: Obstacle | o1.oid != o2.oid
}

fact UniqueReportIDs {
  all disj r1, r2: ObstacleReport | r1.rid != r2.rid
}

fact ObstacleReportsCoherence {
  -- each report must point back to the obstacle, and obstacle.reports
     must match
  all o: Obstacle | all r: o.reports | r.obstacle = o
```

```
all r: ObstacleReport | r in r.obstacle.reports

  -- kind consistent with reports
  all r: ObstacleReport | r.reportedKind = r.obstacle.kind
}

fact ObstacleDecisionCorrectness {
  -- Pending => no decidedBy ; otherwise => exactly one decidedBy
  all o: Obstacle |
    (o.decision = Pending implies no o.decidedBy) and
    (o.decision != Pending implies one o.decidedBy)
}

fact ObstacleDecisionAuthority {
  -- allow either Admin or the owner of the path to decide
  all o: Obstacle |
    (o.decision != Pending implies o.decidedBy in (Admin +
       o.onPath.owner))
}

-- Merge rule:
-- if ANY report confirms -> ConfirmedObs
-- else if ANY report rejects -> RejectedObs
-- else -> Pending
fun mergedDecision[o: Obstacle] : one ObstacleDecision {
  { d: ObstacleDecision |
      (d = ConfirmedObs and ConfirmedObs in o.reports.reportDecision)
   or (d = RejectedObs  and ConfirmedObs not in o.reports.reportDecision
                        and RejectedObs  in o.reports.reportDecision)
   or (d = Pending      and ConfirmedObs not in o.reports.reportDecision
                        and RejectedObs  not in o.reports.reportDecision)
  }
}

fact ObstacleMergedDecisionMatches {
  all o: Obstacle | o.decision = mergedDecision[o]
}

-----------------------------------------------------------------
-- PathStatus
-----------------------------------------------------------------

sig PathStatus {
  sid: one StatusID,
  author: one RegisteredUser,
  target: one Path,
  condition: one PathCondition,
  timestamp: one Int
```

```
}

fact UniqueStatusIDs {
  all disj s1, s2: PathStatus | s1.sid != s2.sid
}

fact UniqueTimestampPerUserPerPath {
  all u: RegisteredUser, p: Path |
    all disj s1, s2: PathStatus |
      (s1.author = u and s1.target = p and s2.author = u and s2.target
         = p) implies
      s1.timestamp != s2.timestamp
}

fact StatusNonRejectedPaths {
  -- Community status makes sense only on public & non-rejected paths
  all ps: PathStatus |
    ps.target.visibility = Public and ps.target.review != Rejected
}

fun statusesOf[p: Path] : set PathStatus { { ps: PathStatus | ps.target
   = p } }

-- Latest status by a given user on a given path
fun latestStatus[u: RegisteredUser, p: Path] : lone PathStatus {
  { s: PathStatus |
      s.author = u and s.target = p and
      all s2: PathStatus |
        (s2.author = u and s2.target = p) implies s2.timestamp <=
           s.timestamp
  }
}

-- Use only "latest per user" for aggregation
fun latestConditions[p: Path] : set PathCondition {
  { c: PathCondition |
      some u: RegisteredUser |
        (some s: latestStatus[u, p] | s.condition = c)
  }
}

-- Unified status (dominance rule): Bad > Caution > Good > none
fun unifiedCondition[p: Path] : lone PathCondition {
  { c: PathCondition |
      (c = Bad     and Bad in latestConditions[p])
   or (c = Caution and Bad not in latestConditions[p]
                   and Caution in latestConditions[p])
   or (c = Good    and Bad not in latestConditions[p]
```

```
                    and Caution not in latestConditions[p]
                    and Good in latestConditions[p])
  }
}


fact PublicationRules {
  -- Published => Public and Confirmed
  all p: Path | (p.publication = Published implies (p.visibility =
     Public and p.review = Confirmed))

  -- Rejected => never Published (BUT can still be referenced by Trip)
  all p: Path | (p.review = Rejected implies p.publication != Published)

  -- Private => Unpublished
  all p: Path | (p.visibility = Private implies p.publication =
     Unpublished)

  -- Archived allowed only from Confirmed
  all p: Path | (p.publication = Archived implies p.review = Confirmed)

  -- If unconfirmed, it must be unpublished
  all p: Path | (p.review = Unconfirmed implies p.publication =
     Unpublished)
}

fun aggregablePaths : set Path {
  { p: Path | p.visibility = Public and p.review = Confirmed and
     p.publication = Published }
}

fun publicObstacles : set Obstacle {
  { o: Obstacle | o.onPath.publication = Published and o.decision =
     ConfirmedObs }
}


-------------------------------------------------------------
-- Search / Filters / Ranking (paths)
-------------------------------------------------------------

-- The user asks the system for candidate paths (route search request)
sig RouteRequest {
  requester: one User,
  candidates: set Path,
  filters: set Filter
}
```

```
-- Filter types
enum FilterKind { OnlyPublicConfirmedPublished, AvoidBad, PreferGood }

sig Filter { kind: one FilterKind }

-- A path satisfies a given filter
pred satisfies[p: Path, f: Filter] {
  (f.kind = OnlyPublicConfirmedPublished) implies
    (p.visibility = Public and p.review = Confirmed and p.publication =
       Published)

  (f.kind = AvoidBad) implies
    (unifiedCondition[p] != Bad)

  (f.kind = PreferGood) implies
    (unifiedCondition[p] = Good)
}

-- Valid candidates are those satisfying all filters
fun validPaths[rq: RouteRequest] : set Path {
  { p: Path |
      p in rq.candidates and
      all f: rq.filters | satisfies[p, f]
  }
}

-- Ranking: a best path is a valid path such that no other valid path
   is "better"
-- (Good > Caution > Bad > none)
pred betterThan[p1, p2: Path] {
  -- p1 is better than p2 if:
  (unifiedCondition[p1] = Good and unifiedCondition[p2] != Good) or
  (unifiedCondition[p1] = Caution and unifiedCondition[p2] = Bad) or
  (some unifiedCondition[p1] and no unifiedCondition[p2])
}

fun bestPaths[rq: RouteRequest] : set Path {
  { p: validPaths[rq] |
      no q: validPaths[rq] | betterThan[q, p]
  }
}

-------------------------------------------------------------
-- Configurable merging policy
-------------------------------------------------------------

-- Policy with weights for each condition
sig MergingPolicy {
```

```
  wGood: one Int,
  wCaution: one Int,
  wBad: one Int
}

-- The single active policy in the system
one sig ActivePolicy { pol: one MergingPolicy }

-- Score of a condition according to the active policy
fun score[c: PathCondition] : one Int {
  { n: Int |
      (c = Good implies n = ActivePolicy.pol.wGood) and
      (c = Caution implies n = ActivePolicy.pol.wCaution) and
      (c = Bad implies n = ActivePolicy.pol.wBad)
  }
}

-- Choose the condition with maximum score among those present in
   latestConditions[p]
fun unifiedConditionByPolicy[p: Path] : lone PathCondition {
  { c: PathCondition |
      c in latestConditions[p] and
      all c2: latestConditions[p] | score[c2] <= score[c]
  }
}

-----------------------------------------------------------------
-- Flags & Moderation (Admin)
-----------------------------------------------------------------

enum FlagReason { Incorrect, Offensive, Spam }

sig PathFlag {
  fid: one Int,
  by: one RegisteredUser,
  target: one Path,
  reason: one FlagReason,
  ts: one Int
}

fact UniqueFlagIDs {
  all disj f1, f2: PathFlag | f1.fid != f2.fid
}

-- A user cannot flag the same path twice at the same timestamp
fact NoDuplicateFlagsSameMoment {
  all u: RegisteredUser, p: Path |
    all disj f1, f2: PathFlag |
```

```
      (f1.by=u and f1.target=p and f2.by=u and f2.target=p) implies
         f1.ts != f2.ts
}

enum ModerationOutcome { Keep, Unpublish, Archive }

sig ModerationAction {
  mid: one Int,
  admin: one Admin,
  target: one Path,
  outcome: one ModerationOutcome,
  ts: one Int
}

fact UniqueModerationIDs {
  all disj m1, m2: ModerationAction | m1.mid != m2.mid
}

-- If a path is moderated with Unpublish/Archive, then it cannot be
   Published
fact ModerationEffectOnPublication {
  all m: ModerationAction |
    (m.outcome = Unpublish or m.outcome = Archive) implies
       m.target.publication != Published
}
```

4.2 Assertions

The following assertions are used to validate critical safety and consistency properties of the model. Each assertion is checked within a bounded scope in order to automatically search for counterexamples and highlight potential inconsistencies.

- **`PublicObstacleValidity`.** This assertion verifies that every obstacle included in `publicObstacles` is actually eligible to be shown to end users. In particular, a public obstacle must be `ConfirmedObs` and it must belong to a path that is `Published`, `Confirmed` (review state), and `Public` (visibility). This prevents the exposure of unreviewed paths, private paths, or undecided obstacles through the "public" obstacle set.

- **`ModerationBlocksPublication`.** This assertion checks the effect of administrative moderation on publication. If a path is the target of any moderation action with outcome `Unpublish` or `Archive`, then the path cannot remain in the `Published` state. The property ensures that moderation decisions are reflected in the publication lifecycle, preventing configurations where content is both moderated and still publicly published.

- **`BestPathsCorrect`.** This assertion validates the correctness of the route search ranking logic. For a given `RouteRequest`, it ensures that:

- every element returned by `bestPaths[rq]` is a valid candidate, i.e., it satisfies all filters (`bestPaths` is a subset of `validPaths`);
- no other valid path is strictly better than any selected best path according to the `betterThan` relation.

```
assert PublicObstacleValidity {
  all o: Obstacle |
    (o in publicObstacles) implies
      (o.decision = ConfirmedObs and
       o.onPath.publication = Published and
       o.onPath.review = Confirmed and
       o.onPath.visibility = Public)
}

check PublicObstacleValidity for 10


assert ModerationBlocksPublication {
  all p: Path |
    (some m: ModerationAction | m.target = p and (m.outcome = Unpublish
       or m.outcome = Archive))
    implies
    p.publication != Published
}

check ModerationBlocksPublication for 10


assert BestPathsCorrect {
  all rq: RouteRequest |
    some rq implies
      (all p: bestPaths[rq] | p in validPaths[rq]) and
      (all p: bestPaths[rq] | no q: validPaths[rq] | betterThan[q, p])
}

check BestPathsCorrect for 6
  but exactly 1 RouteRequest,
      exactly 2 Path,
      exactly 1 Filter,
      exactly 2 RegisteredUser,
      exactly 1 Admin,
      exactly 1 PathStatus,
      exactly 0 Trip,
      exactly 0 Obstacle,
      exactly 0 ObstacleReport
```

4.2.1 Results of Assertions

The following figures report the outcome produced by Alloy Analyzer when executing the assertions.

```
Executing "Check PublicObstacleValidity for 10"
  Actual scopes: 10 UID, 10 PathID, 10 TripID, 10 ObstacleID, 10
  Solver=sat4j Bitwidth=4 MaxSeq=7 SkolemDepth=1 Symmetr
  106273 vars. 4240 primary vars. 156072 clauses. 341ms.
  No counterexample found. Assertion may be valid. 48ms.

Executing "Check ModerationBlocksPublication for 10"
  Actual scopes: 10 UID, 10 PathID, 10 TripID, 10 ObstacleID, 10
  Solver=sat4j Bitwidth=4 MaxSeq=7 SkolemDepth=1 Symmetr
  105916 vars. 4250 primary vars. 155436 clauses. 332ms.
  No counterexample found. Assertion may be valid. 67ms.

Executing "Check BestPathsCorrect for 6 but exactly 1 Route
  Actual scopes: 6 UID, 6 PathID, 6 TripID, 6 ObstacleID, 6 Status
  Solver=sat4j Bitwidth=4 MaxSeq=6 SkolemDepth=1 Symmetr
  0 vars. 0 primary vars. 3587ms.
  No counterexample found. Assertion may be valid. 0ms.
```

Figure 17: Assertions results

.

4.3 Predicates

To complement the structural constraints of the model, three predicates are executed to generate meaningful instances of the BBP domain. Each predicate targets a specific subsystem (paths/trips, obstacles, and routing) and is paired with a bounded `run` command to keep the generated worlds compact and easy to inspect.

- **`baseWorld`.** This predicate generates a minimal consistent configuration of the system. It enforces the presence of exactly one administrator, two registered users, and one guest, together with a single trip and a single path. Moreover, it guarantees that at least one path is manually created and that there exists a path already in a fully publishable state (public, confirmed, and published). This scenario is useful as a baseline to verify that the core entities and lifecycle constraints can coexist in a small instance.

- **`obstacleWorld`.** This predicate focuses on obstacle reporting and decision merging. It requires at least one path that is publicly available (public, confirmed, published) and at least one confirmed obstacle placed on that path, supported by multiple reports. The corresponding `run` command further restricts the scope (one path, one obstacle, exactly two reports, one trip, and no route requests or path statuses) in order to obtain a compact instance that highlights the obstacle/report subsystem without introducing unrelated elements.

- **routingWorld.** This predicate models a simplified route search scenario. It requires the existence of a route request with at least two candidate paths, at least one filter, and at least one path selected by `bestPaths`. In addition, it enforces the presence of community feedback by requiring at least one path status and a defined aggregated condition (through `unifiedCondition`). The `run` command fixes the instance size (one request, two paths, one filter, one status, and no trips, obstacles, or moderation objects) so that the generated world isolates the filtering and ranking logic used during route search.

```
pred baseWorld {
  #Admin = 1
  #RegisteredUser = 2
  #Guest = 1

  #Trip = 1
  #Path = 1
  some ManualPath
  some p: Path | p.visibility = Public and p.review = Confirmed and
     p.publication = Published
}

run baseWorld for 5



pred obstacleWorld {
  #RegisteredUser = 2
  #Admin = 1

  some p: Path |
    p.visibility = Public and
    p.review = Confirmed and
    p.publication = Published and
    some o: Obstacle |
      o.onPath = p and
      o.decision = ConfirmedObs and
      #o.reports >= 2
}

run obstacleWorld for 6
  but exactly 0 Guest,
      exactly 1 Path,
      exactly 1 Obstacle,
      exactly 2 ObstacleReport,
      exactly 0 PathStatus,
      exactly 0 RouteRequest,
      exactly 1 Trip
```

```
pred routingWorld {
  some rq: RouteRequest |
    #rq.candidates >= 2 and
    some rq.filters and
    some bestPaths[rq]

  some p: Path |
    some statusesOf[p] and
    some unifiedCondition[p]
}

run routingWorld for 5
but exactly 1 RouteRequest,
      exactly 2 Path,
      exactly 1 Filter,
      exactly 2 RegisteredUser,
      exactly 1 Admin,
      exactly 0 Guest,
      exactly 1 PathStatus,
      exactly 0 Obstacle,
      exactly 0 ObstacleReport,
      exactly 0 PathFlag,
      exactly 0 ModerationAction,
      exactly 0 Trip
```

4.3.1 Results

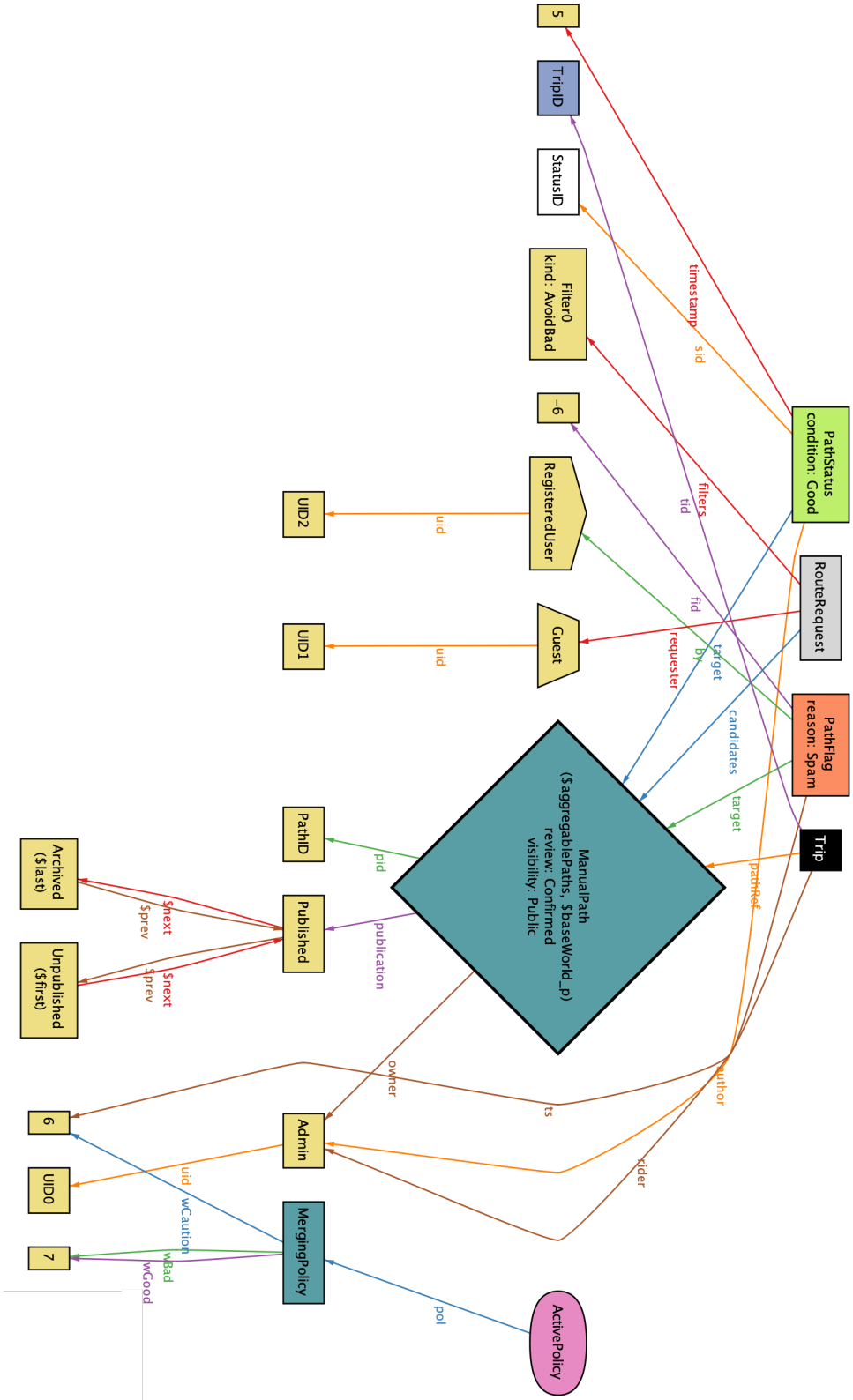


Figure 18: Instance for `baseWorld`.

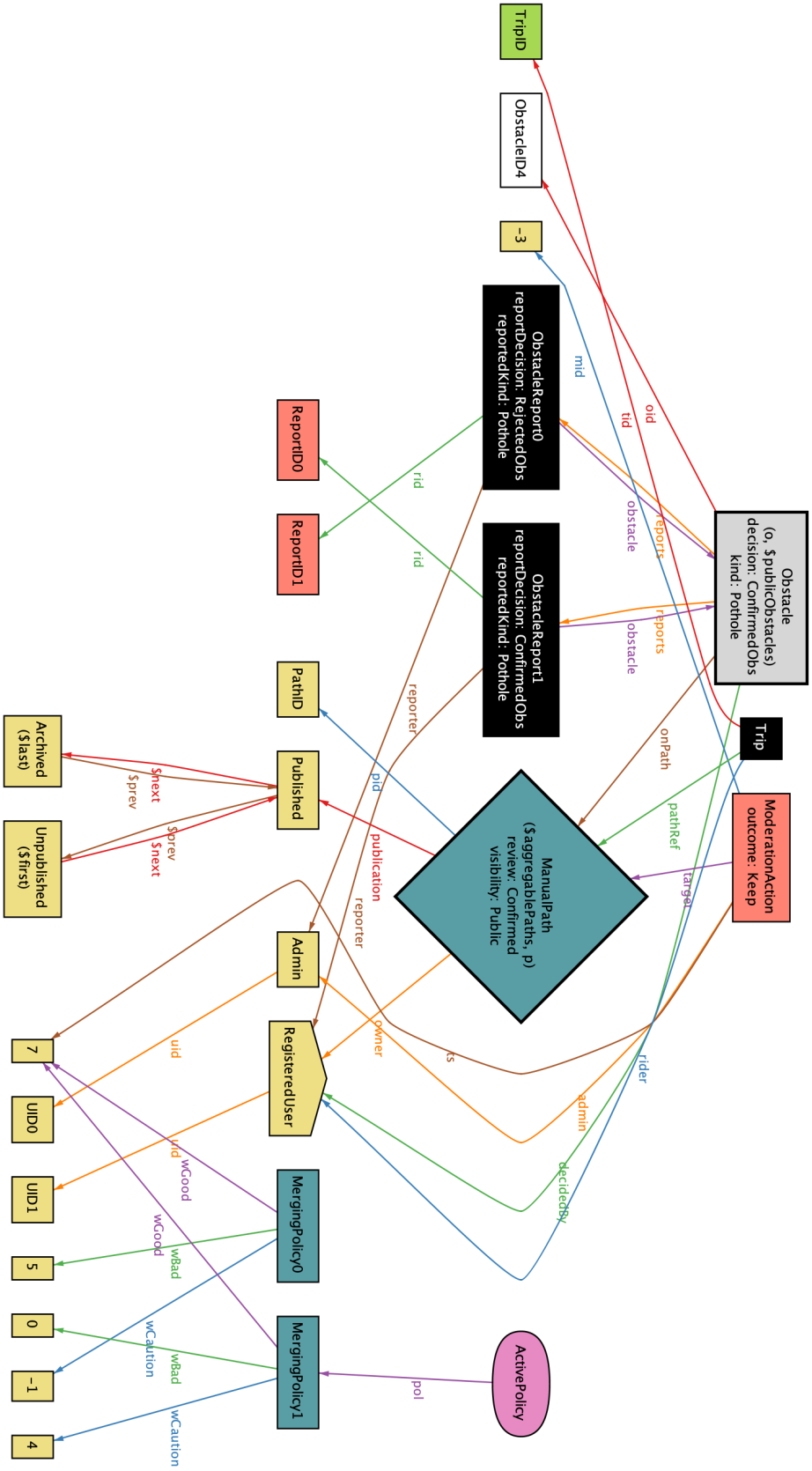


Figure 19: Instance for `obstacleWorld`.

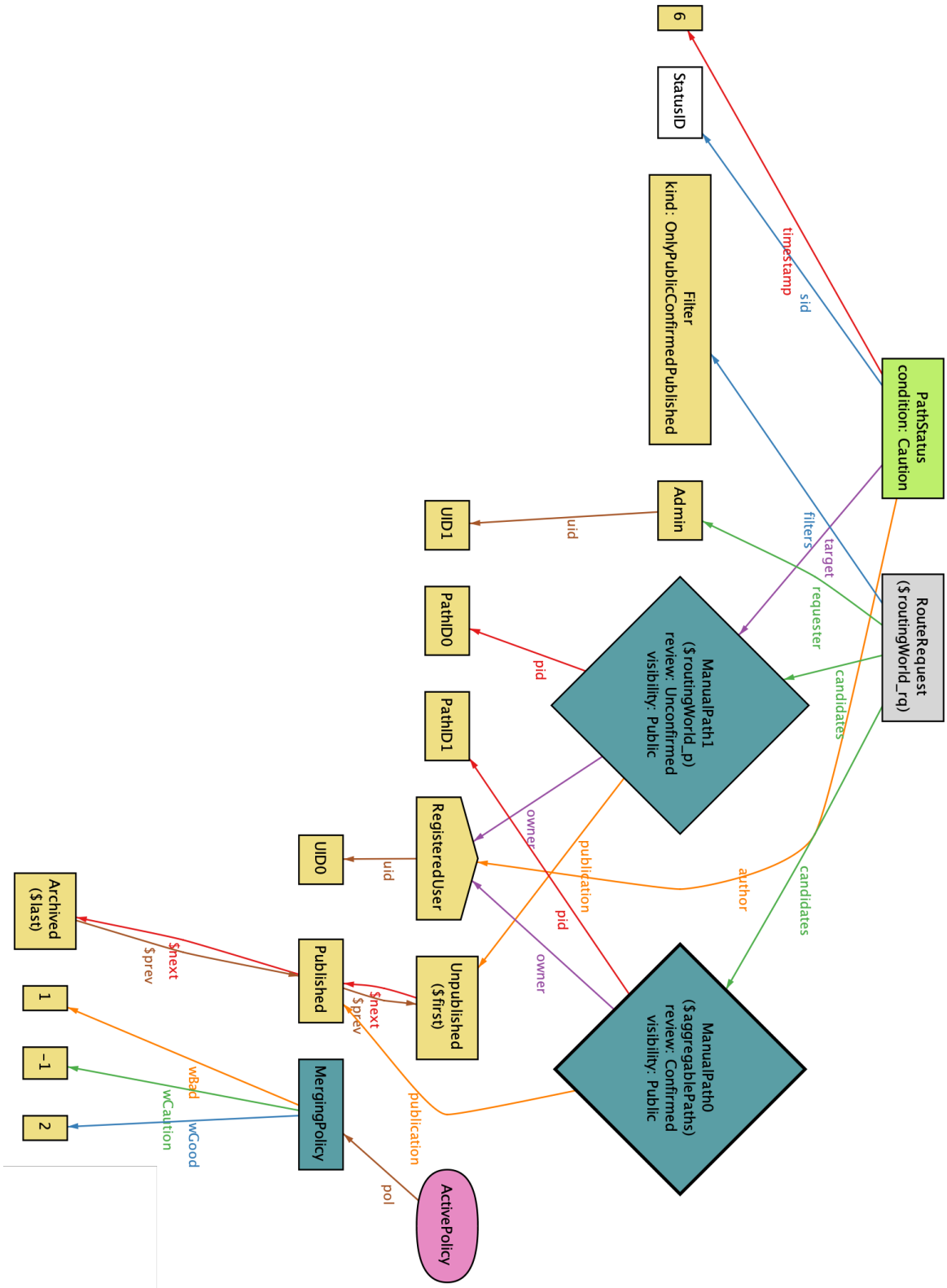


Figure 20: Instance for `routingWorld`.

5 Effort spent

The table below shows the time spent by each team member on the project.

	Introduction	Overall desc.	Specific req.	Analysis u. alloy
Wojciech Maciej Bukala	2	14	24	6
Giacomo Rigo	2	9	20	15
Mattia Zonzin	6	10	25	5

Table 1: Table of effort spent